Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

CONTENTS

Table of contents

Each module starts on its own opener page. Page numbers restart inside each module and are shown in that module footer.

Module 1. Version Control and Git Basics

- 1.1 What is version control?
- 1.2 Git vs GitHub vs VS Code
- 1.3 Understanding the Git workflow
- 1.4 Lab and quiz

Module 2. Installing and Configuring Git

- 2.1 Installing Git and VS Code
- 2.2 Creating your GitHub account
- 2.3 First-time Git configuration
- 2.4 Lab and quiz

Module 3. Creating Your First Repository

- 3.1 Understanding repositories
- 3.2 git init and git status
- 3.3 Tracking your first files
- 3.4 Lab and quiz

Module 4. Git Add, Commit, and History

- 4.1 Understanding git add
- 4.2 Understanding git commit
- 4.3 Reviewing Git history
- 4.4 Lab and quiz

Module 5. Connecting Git to GitHub

- 5.1 Local vs remote repositories
- 5.2 git remote and git push
- 5.3 Exploring a GitHub repository
- 5.4 Lab and quiz

Module 6. Git and GitHub in VS Code

- 6.1 VS Code Source Control
- 6.2 Commit and push without the terminal
- 6.3 Branches and history in VS Code
- 6.4 Lab and quiz

Module 7. Branches and Merging

- 7.1 Understanding branches
- 7.2 Creating and merging branches
- 7.3 Merge conflicts
- 7.4 Lab and quiz

Module 8. Clone, Fetch and Pull

- 8.1 Cloning a repository
- 8.2 Fetching changes
- 8.3 Pulling changes
- 8.4 Lab and quiz

Module 9. Collaboration and Pull Requests

- 9.1 The branch-and-push workflow
- 9.2 Opening a pull request
- 9.3 Review, merge and clean up
- 9.4 Lab and quiz

Module 10. Undoing Mistakes Safely

- 10.1 Undo changes before you commit
- 10.2 Fix or revert a commit
- 10.3 Reset, reflog and common errors
- 10.4 Lab and quiz

Module 11. Professional Habits

- 11.1 Writing a README with Markdown
- 11.2 Ignoring files with .gitignore
- 11.3 Security and good habits
- 11.4 Lab and quiz

Module 12. Final Project: Portfolio Website

- 12.1 Plan and start the project
- 12.2 Build with branches and pull requests
- 12.3 Publish with GitHub Pages and submit
- 12.4 Lab and quiz

Resources

- Final project and rubric
- Git cheat sheet
- Glossary
- Troubleshooting guide
- Screenshot checklist

BEFORE YOU START

About this course

Who it is for

Beginners who have never used Git. You need a Windows computer and an internet connection.

How the course works

- 12 modules, each with 3 lessons, a practical lab and a 5-question quiz (pass mark 70%).
- You practise on a project called my-first-project, then build a portfolio website.
- Commands run in Git Bash; the VS Code equivalents are shown where they exist.
- Figures are labeled illustrations, not real screenshots.

Conventions

Code and commands are always written left to right and in a monospaced font. Git terms stay in English in the Arabic edition.

01

MODULE 01

Understanding Git & GitHub

What version control is, why every developer uses it, and how Git, GitHub and VS Code fit together.

MODULE 01 · LESSON 1.1

What is version control?

ما هو التحكم بالإصدارات؟

Learning objectives

- Explain what version control is in your own words.
- Describe three problems caused by copying project folders by hand.
- Explain how Git records changes as a chain of commits.
- Tell the difference between local and distributed version control.

Prerequisites

None. You only need a computer and a project, even a simple text file, that you have edited more than once.

Introduction

Imagine you are writing an important report. On Monday you save `report.docx`. On Tuesday you change it and save `report_final.docx`. By Friday you have `report_final2.docx` and `report_REAL_final.docx`, and you are not sure which one is right. Version control solves exactly this problem, and it does it for every file in a project at once.

Explanation

Version control is a system that records every change made to a set of files, who made it, when, and why. You can go back to any earlier moment, compare two moments, and work with other people without overwriting each other.

Why developers need it. Code changes constantly. A new feature can break something that worked yesterday. With version control you can see exactly what changed and undo it safely. Teams also need it: five people can edit the same project and combine their work.

The problem with copying folders. Copies take space, names become confusing, nothing explains *why* a copy exists, and two people copying the same folder can never merge their changes automatically.

How Git works. Git does not keep random copies. When you tell it to, it saves a **commit**: a snapshot of your project plus a message and an author. Each commit points to the one before it, so your history is a chain you can walk backwards.

Local and distributed. Older systems kept the history on one central server. Git is **distributed**: every developer has the full history on their own computer. You can work offline, and if a server is lost, any copy can restore it.

Real-world analogy

> Think of a video game with save slots. Each save is a moment you can return to. A commit is a save point for your project, and the message is the label you write on it, such as "Before the boss fight".

Diagram

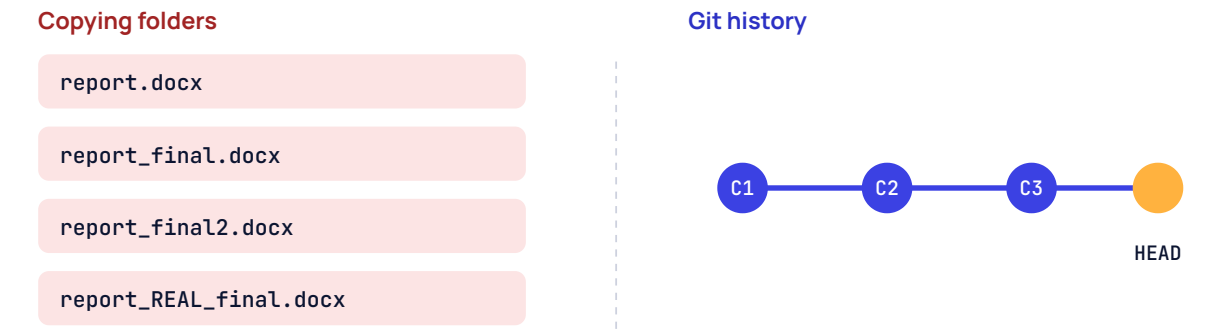


Figure 1 · Copying folders vs. a Git history

Commands

There are no commands in this lesson. In Lesson 2.1 you will install Git and run your first one, `git --version`.

Common mistakes and solutions

"Git is a backup tool."	Git records meaningful changes you choose to save. Keep real backups too.
"I must be online to use Git."	Git works fully offline. You only need internet to share with GitHub.
"Version control is only for big teams."	It helps a single person from the first day of a project.

Practical exercise

> Find a project or document you have edited many times. Write down three situations where you wished you could go back or compare versions. For each one, note which Git idea (history, commit, or sharing) would help.

Expected result

A short list of three real situations, each matched to an idea from this lesson. There is no right or wrong list. It prepares you for Lesson 1.2.

Knowledge check

1. What does a commit store?

- ○ A copy of the internet
- ○ A snapshot of the project, a message, and an author
- ○ Only the names of changed files

2. Why is Git called distributed?

- ○ Every developer has the full history locally
- ○ It only runs on one server
- ○ It distributes free files

3. Which is a problem of copying folders by hand?

- ○ Windows deletes the copies
- ○ Unclear names and no explanation of changes
- ○ Files become read-only

Answers

- A snapshot of the project, a message, and an author — A commit is a snapshot with a message and author, linked to the commit before it.
- Every developer has the full history locally — Each copy contains the whole history, so work does not depend on one central server.

- Unclear names and no explanation of changes — Names like final2 do not say what changed or why, and copies cannot be merged automatically.

Summary

Version control records the history of a project. Git saves snapshots called commits, keeps the full history on every computer, and lets you return to any point or combine work from several people.

Key vocabulary

Version control	A system that records changes to files over time
Commit	A saved snapshot with a message and author
History	The chain of commits in a project
Distributed	Every copy holds the full history

Next lesson

> **Lesson 1.2 — Git vs GitHub vs VS Code.** You now know what Git does. Next you will meet the three tools you will use and see how they work together.

MODULE 01 · LESSON 1.2

Git vs GitHub vs VS Code

Git مقابل GitHub مقابل VS Code

Learning objectives

- Define Git, GitHub and VS Code in one sentence each.
- Explain local and remote repositories.
- Tell public repositories from private ones.

Prerequisites

Lesson 1.1.

Introduction

Beginners often use “Git” and “GitHub” as if they were one thing. They are three different tools that work together, and knowing the difference removes most early confusion.

Explanation

Git is a free program on your computer. It records the history of your files. It works without internet.

GitHub is a website that stores Git projects online. It adds sharing and teamwork: backups, issues, pull requests and a profile page.

VS Code (Visual Studio Code) is a code editor. It has Git built in, so you can stage, commit and push with buttons instead of typing.

A **repository** (repo) is a project folder plus its Git history. Your copy is the **local repository**. The copy on GitHub is the **remote repository**. A **commit** is one saved snapshot, and a **branch** is a line of commits. You will study branches in Module 7.

Public repositories can be seen by anyone. **Private** repositories are visible only to you and people you invite. Never put passwords in either kind.

Real-world analogy

> Writing a book: VS Code is your writing desk, Git is the editor who keeps every draft, and GitHub is the shared cloud folder your team reads.

Diagram

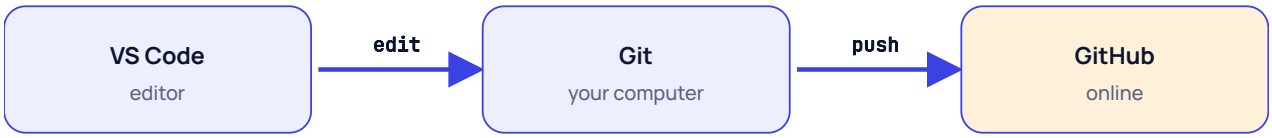


Figure 1 · How the three tools fit together

Commands

No commands yet. In Lesson 2.1 you will install the tools and run `git --version`.

Common mistakes and solutions

Thinking GitHub is required to use Git.	Git works alone. GitHub is optional, but very useful for sharing.
Thinking VS Code and Git are the same.	VS Code is an editor. It only gives you buttons for Git.
Making a repo public by accident.	Check the visibility setting before you create or publish a repository.

Practical exercise

> Open github.com in your browser. Open any public repository. Find three things: the README text, the list of files, and the branch name (usually `main`).

Expected result

You can point to the README, the file list and the branch selector on the page. You did not need an account for this.

Knowledge check

1. Which tool stores your project online?

- ○ Windows Terminal
- ○ GitHub
- ○ VS Code

2. Where is the local repository?

- ○ On your computer
- ○ Only on GitHub
- ○ Inside the browser

3. Who can see a private repository?

- ○ Only GitHub staff
- ○ Only you and people you invite
- ○ Everyone

Answers

- GitHub — GitHub hosts remote repositories.
- On your computer — The local repository is the project history stored on your own machine.
- Only you and people you invite — Private means visible only to you and chosen collaborators.

Summary

Git tracks changes, GitHub hosts repositories online, and VS Code is the editor with Git buttons. Your computer holds the local repository, GitHub holds the remote one.

Key vocabulary

Repository	A project folder with its Git history
Local / Remote	On your computer / on GitHub
Public / Private	Visible to everyone / only to invited people
Branch	A line of commits

Next lesson

> **Lesson 1.3 — The Git workflow.** Next you will follow a file from your editor to GitHub, stage by stage.

MODULE 01 · LESSON 1.3

Understanding the Git workflow

فهم سير عمل Git

Learning objectives

- Name the four places a change can live.
- Recognize untracked, modified, staged and committed files.
- Explain add → commit → push.

Prerequisites

Lessons 1.1 and 1.2.

Introduction

Git does not save everything automatically. You choose what to save and when. That choice happens in stages, and learning them makes every later command easy to understand.

Explanation

Working directory: your project folder, where you edit files.

Untracked files are new files Git has never seen. **Modified** files are tracked files you changed since the last commit.

Staging area: a waiting room. You place the changes you want in the next commit here. The command is `git add`.

Local repository: the history on your computer. `git commit` saves the staged changes as a new commit.

Remote repository: the copy on GitHub. `git push` sends your new commits there.

The **commit history** is the list of all commits. **Remember:** staging is not committing, and committing is not pushing. Each is a separate step.

Real-world analogy

> Packing a parcel: you edit in your room (working directory), put the items you chose into the box (staging), seal and label it (commit), then send it by post (push).

Diagram

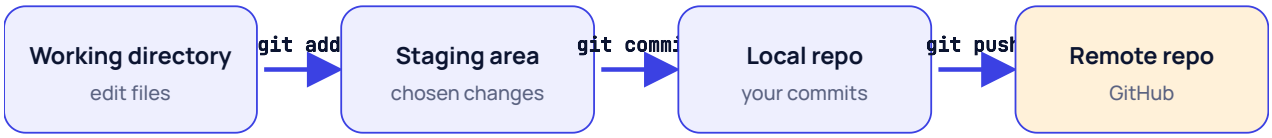


Figure 1 · The add → commit → push workflow

Commands

You will learn these commands in Modules 4 and 5. For now, just recognize them: `git add`, `git commit -m "message"`, `git push`.

Common mistakes and solutions

Thinking add saves the work.	It only prepares changes. Nothing is saved until you commit.
Thinking commit uploads to GitHub.	Commit saves on your computer only. Push uploads.
Expecting new files to be tracked automatically.	New files are untracked until you add them.

Practical exercise

> Imagine you edit `index.html`, then run `git add index.html`, then `git commit -m "Add homepage"`. Write which stage the change is in after each of the three actions.

Expected result

After editing: modified, in the working directory. After add: staged. After commit: saved in the local repository, not yet on GitHub.

Knowledge check

1. Which command sends commits to GitHub?

- ○ git commit
- ○ git push
- ○ git add

2. A brand-new file Git has never seen is…

- ○ Untracked
- ○ Committed
- ○ Pushed

3. Does git add create a commit?

- ○ Only on Fridays
- ○ No, it only stages changes
- ○ Yes

Answers

- git push — push uploads commits to the remote repository.
- Untracked — New files are untracked until you add them.
- No, it only stages changes — Staging prepares changes. Commit creates the snapshot.

Summary

A change travels from the working directory to the staging area (add), to the local repository (commit), and to the remote repository (push).

Key vocabulary

Working directory	Your project folder
Staging area	Where changes wait for the next commit
Untracked / Modified	New file / changed tracked file
Push	Send commits to the remote

Next lesson

Module 2 — Installing and configuring Git. You know the theory. Now you will install the tools and run your first command.

MODULE 01

Practical lab 1 · Identify every stage

Read the scenario and fill in the table. Scenario: you create `style.css`, edit `index.html`, then run `git add index.html`, `git commit -m "Add homepage"`, and `git push`.

Moment	Where is `index.html`'s change?	Where is `style.css`?
After editing	Modified (working directory)	Untracked
After git add	Staged	Untracked
After git commit	Local repository	Untracked
After git push	Remote repository (GitHub)	Untracked

Evaluation: 4 correct rows = pass. Note that `style.css` stays untracked because it was never added.

Module 1 quiz (pass mark: 70%)

1. What is Git?

- ○ A website
- ○ A code editor
- ○ A program that tracks file history

2. What is GitHub?

- ○ A file format
- ○ A website that hosts Git repositories
- ○ A terminal

3. What does staging do?

- ○ Prepares changes for the next commit
- ○ Uploads to GitHub
- ○ Deletes files

4. After git commit, where are your changes?

- ○ Nowhere
- ○ In the local repository
- ○ On GitHub

5. Can Git work without internet?

- ○ No
- ○ Only on Windows
- ○ Yes

Answers

- A program that tracks file history — Git is the version control program on your computer.
- A website that hosts Git repositories — GitHub stores remote repositories online.
- Prepares changes for the next commit — Staging selects the changes for the next commit.
- In the local repository — Commit saves on your computer; push is the step that uploads.
- Yes — Git is distributed; the history is local.

02

MODULE 02

Installing and Configuring Git

Install Git and VS Code, create your GitHub account, and tell Git who you are.

MODULE 02 · LESSON 2.1

Installing Git and VS Code

تثبيت Git وVS Code

Learning objectives

- Install Git on Windows.
- Install Visual Studio Code.
- Open the integrated terminal and choose Git Bash.
- Verify the installation with `git --version`.

Prerequisites

Module 1. A Windows computer and an internet connection.

Introduction

Before any command works, the tools must be on your computer. This lesson installs both, then checks that Git answers.

Explanation

Install Git. Go to `git-scm.com` and choose **Download for Windows**. Run the installer. The default options are safe for beginners. You may choose Visual Studio Code as the default editor, and choose **main** as the name for new branches if the installer offers it. Click Next until Install, then Finish.

Verify. Open **Git Bash** from the Start menu and run `git --version`. If Git answers with a version number, it is installed.

Install VS Code. Go to `code.visualstudio.com`, download the Windows installer and run it. Keep **Add to PATH** enabled.

Open the terminal in VS Code. Use the menu **Terminal → New Terminal**, or press `Ctrl+`` . In the terminal panel, use the dropdown arrow next to the **+** and choose **Git Bash** as the profile.

Real-world analogy

> Installing tools is like stocking a kitchen before you cook: nothing to cook yet, but every recipe in this course needs these tools.

Diagram

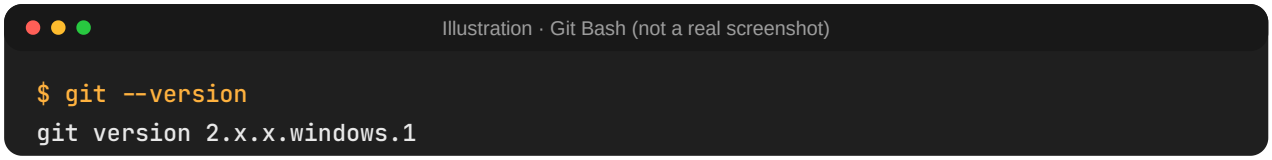


Figure 1 · Illustration of the expected output

Commands

Run this in Git Bash or the VS Code terminal. `--version` is an option that asks the program to print its version and exit.

```
git --version
```

Common mistakes and solutions

“git is not recognized”	Close and reopen the terminal. If it still fails, run the Git installer again.
Opening PowerShell and expecting Bash commands.	Choose Git Bash in the terminal profile list.
Skipping Add to PATH for VS Code.	Re-run the installer and enable it.

Practical exercise

> Install Git and VS Code, open a terminal inside VS Code, select Git Bash, and run `git --version`.

Expected result

The terminal prints a line starting with `git version`.

Knowledge check

1. Which command checks the Git installation?

- ○ git check
- ○ git –version
- ○ git start

2. Which terminal do we use in this course?

- ○ Git Bash
- ○ Notepad
- ○ Paint

3. Which shortcut opens a terminal in VS Code?

- ○ Alt+F4
- ○ Ctrl+`
- ○ Ctrl+Z

Answers

- git –version — `git --version` prints the installed version.
- Git Bash — Git Bash gives us the same commands on Windows.
- Ctrl+` — Ctrl+` toggles the integrated terminal.

Summary

You installed Git and VS Code, opened a terminal in VS Code, chose Git Bash, and confirmed Git works.

Key vocabulary

Terminal	A text window for typing commands
Git Bash	A Bash terminal that comes with Git for Windows
PATH	The list of folders Windows searches for programs

Next lesson

Lesson 2.2 — Creating your GitHub account. The tools are ready. Next you create your online home.

MODULE 02 · LESSON 2.2

Creating your GitHub account

إنشاء حساب GitHub

Learning objectives

- Create and verify a GitHub account.
- Find your way around the dashboard.
- Secure the account with two-factor authentication.
- Name the ways Git signs in to GitHub.

Prerequisites

Lesson 2.1 and an email address you can open.

Introduction

Your GitHub account is your identity online. Your projects, your history and your future portfolio live there, so we protect it from day one.

Explanation

Create the account. Go to `github.com` and choose **Sign up**. Enter your email, a strong password and a username. Choose a username you would be happy to show an employer.

Verify your email. GitHub sends a code to your email. Enter it to finish. You may also be asked to solve a short puzzle.

The dashboard. After sign-in you see your repositories, activity and suggestions. Your avatar (top right) opens the menu with **Your profile**, **Your repositories** and **Settings**.

Profile. Add a photo and a short bio. Keep personal details you do not want public out of it.

Security. Use a unique password in a password manager. In **Settings → Password and authentication** enable **two-factor authentication (2FA)** with an authenticator app, and save the recovery codes somewhere safe.

How Git signs in. Over **HTTPS**, Git asks for your GitHub login through a sign-in window and keeps it in Git Credential Manager (included with Git for Windows). With **SSH**, you add a key pair to your account. Never use your GitHub password as a Git password in the terminal. You will set up a method in Module 5.

Real-world analogy

> Your account is like a bank account for your code: a strong password and a second lock (2FA) keep it yours.

Diagram

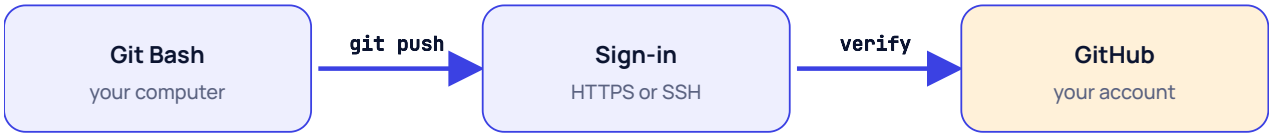


Figure 1 · Git proves who you are before it uploads

Commands

No Git commands here. All steps happen in the browser.

Common mistakes and solutions

Losing the 2FA recovery codes.	Store them in a password manager or print them.
Using a private email in public commits.	Later you can use the GitHub “noreply” address (Lesson 2.3).
Choosing a joke username.	Pick a professional one. It becomes part of your URLs.

Practical exercise

> Create your account, verify the email, add a profile photo, and turn on 2FA.

Expected result

You can open your profile at `github.com/your-username` and Settings shows 2FA as enabled.

Knowledge check

1. Why enable 2FA?

- ○ It is required to install Git
- ○ A second check protects the account
- ○ It makes Git faster

2. Where do your projects live online?

- ○ GitHub
- ○ VS Code
- ○ Git Bash

3. Name one way Git signs in to GitHub.

- ○ Screenshots
- ○ SSH keys
- ○ Email attachments

Answers

- A second check protects the account — 2FA stops access with only a stolen password.
- GitHub — GitHub hosts your remote repositories.
- SSH keys — HTTPS with Credential Manager and SSH keys are the two common methods.

Summary

You created and secured a GitHub account and learned the two sign-in methods Git uses: HTTPS and SSH.

Key vocabulary

2FA	A second proof of identity at sign-in
HTTPS / SSH	Two ways Git connects to GitHub
Dashboard	Your home page on GitHub

Next lesson

Lesson 2.3 — First-time Git configuration. Now you tell Git who you are.

MODULE 02 · LESSON 2.3

First-time Git configuration

إعداد Git لأول مرة

Learning objectives

- Set your name, email and default branch.
- Check your settings.
- Explain global and local configuration.
- Separate commit identity from GitHub sign-in.

Prerequisites

Lessons 2.1 and 2.2.

Introduction

Every commit records who made it. Git cannot guess this, so you set it once.

Explanation

Set your identity. Run the first two commands below with your own details. Use the email of your GitHub account, or the private "noreply" address that GitHub shows in **Settings → Emails**.

Default branch. The third command makes new repositories start with a branch named `main`.

Check. `git config --global --list` prints every global setting.

Global vs local. `--global` saves in your user file and applies to all repositories. Without it, inside a repository, a setting is **local** and applies only there. Local wins over global.

Identity vs sign-in. Name and email are only labels stamped on commits. They do not prove who you are. **Authentication** (HTTPS login or SSH key) is what lets you push. You can use SSH later: `ssh-keygen -t ed25519 -C "you@example.com"` creates a key pair. We finish that in Module 5.

Real-world analogy

> The name and email are the signature on a letter. Anyone can write a signature, but only your key opens the mailbox.

Diagram

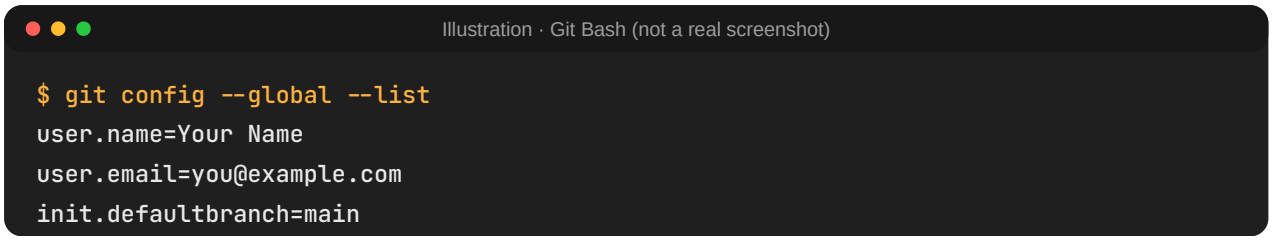


Figure 1 · Illustration of the expected output

Commands

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global --list
```

Options: `--global` user-wide setting; `--list` show settings.

Common mistakes and solutions

Typing the quotes wrongly.	Use straight quotes around names with spaces.
Using a different email than GitHub.	Commits may not link to your profile. Use your GitHub email.
Confusing config with login.	Config only labels commits. Sign-in is separate.

Practical exercise

> Run the four commands with your details, then check the output matches what you typed.

Expected result

The list shows your `user.name`, `user.email` and `init.defaultbranch=main`.

Knowledge check

1. What does user.name do?

- ○ Renames the project
- ○ Labels your commits with your name
- ○ Logs you in to GitHub

2. Which wins, local or global?

- ○ Local
- ○ Global
- ○ Neither

3. Does user.email authenticate you?

- ○ Only on Windows
- ○ No, authentication is separate
- ○ Yes

Answers

- Labels your commits with your name — It is stamped on each commit.
- Local — A local setting overrides a global one in that repository.
- No, authentication is separate — It is a label. HTTPS login or SSH key authenticates.

Summary

You set your name, email and default branch, verified them, and learned that commit identity is not the same as GitHub sign-in.

Key vocabulary

Global / Local	All repos / one repo
Authentication	Proving who you are to GitHub
SSH key	A key pair used instead of a password

Next lesson

Module 3 — Creating your first repository. With Git configured, you will create your first project.

MODULE 02

Practical lab 2 · Configure and verify Git

1. Open Git Bash and run `git --version`.
2. Run the three `git config --global` commands with your details.
3. Run `git config --global --list`.
4. Check each line against what you typed.

Check	Pass if…
Version	Output starts with `git version`
Identity	Name and email match your GitHub account
Branch	`init.defaultbranch=main` appears

Evaluation: 3 of 3 checks = pass.

Module 2 quiz (pass mark: 70%)

1. Which command prints your global settings?

- ○ git show config
- ○ git list
- ○ git config –global –list

2. Which tool is the editor with built-in Git?

- ○ Git Bash
- ○ VS Code
- ○ GitHub

3. What should you do with 2FA recovery codes?

- ○ Store them safely
- ○ Share them
- ○ Delete them

4. What does –global mean?

- ○ Uploads to GitHub
- ○ Applies to all your repositories
- ○ Applies to one file

5. Does user.name prove who you are?

- ○ Yes
- ○ Only with 2FA
- ○ No, it is only a label

Answers

- git config –global –list — `--list` prints the settings.
- VS Code — VS Code includes Git buttons.
- Store them safely — They restore access if you lose your device.
- Applies to all your repositories — It saves in your user-level Git file.
- No, it is only a label — Authentication proves identity; name is a label.

03

MODULE 03

Creating Your First Repository

Turn a plain folder into a Git repository and watch Git notice your first files.

MODULE 03 · LESSON 3.1

Understanding repositories

فهم المستودعات

Learning objectives

- Define a repository.
- Tell a project folder from a Git repository.
- Explain what the `.git` folder holds.
- Tell local from remote repositories.

Prerequisites

Module 2 completed.

Introduction

You have heard the word “repository” many times. Here it gets a precise meaning, and you will see exactly what makes a folder special to Git.

Explanation

A **repository** (repo) is a project folder that Git is tracking, together with the full history of that project.

A **normal folder** is just files. It becomes a **Git repository** when you run `git init` inside it. Git then creates a hidden folder named `.git`.

The **.git directory** stores the commits, branches and settings of the repository. Your own files live next to it, not inside it. **Never edit or delete .git by hand.** Deleting it removes the history, although your files stay.

A **local repository** is the one on your computer. A **remote repository** is a copy on a server such as GitHub. You will connect them in Module 5.

To see hidden items in Windows, open File Explorer and choose **View → Show → Hidden items**.

Real-world analogy

> A folder is a house. The `.git` folder is the archive of security-camera recordings: the house looks the same, but the archive remembers everything that happened inside.

Diagram

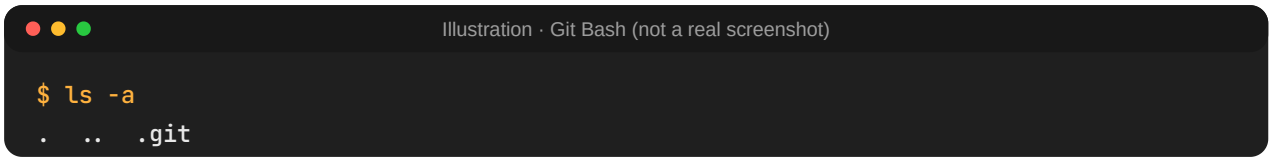


Figure 1 · Illustration: a repository contains a hidden .git folder

Commands

In Git Bash, `ls -a` lists all files, including hidden ones. Inside a repository you will see `.git`.

```
ls -a
```

Common mistakes and solutions

Deleting the .git folder.	Your history is gone. Only do this if you truly want to stop using Git in that project.
Thinking .git is inside your project files.	It is a hidden sibling of your files. Your files stay outside it.
Creating a repo inside another repo.	Check you are in the right folder before `git init`.

Practical exercise

> Turn on hidden items in File Explorer. Open your Documents folder and check that it does not contain a `.git` folder.

Expected result

You see no `.git` folder. Ordinary folders are not repositories until you initialize them (Lesson 3.2).

Knowledge check

1. What makes a folder a Git repository?

- ○ Its name
- ○ A hidden .git folder inside it
- ○ A README file

2. What does .git store?

- ○ Commits, branches and settings
- ○ Your passwords
- ○ Windows updates

3. Where is a remote repository?

- ○ In the Recycle Bin
- ○ On a server such as GitHub
- ○ Inside .git

Answers

- A hidden .git folder inside it — git init creates .git, which holds the history.
- Commits, branches and settings — It holds the repository data.
- On a server such as GitHub — Remote means a copy hosted elsewhere.

Summary

A repository is a project folder plus its Git history, stored in the hidden .git folder. Local is on your computer, remote is on a server such as GitHub.

Key vocabulary

Repository	A project tracked by Git with its history
.git	Hidden folder holding the history and settings
Local / Remote	On your computer / on a server

Next lesson

Lesson 3.2 — git init and git status. Next you create a repository yourself.

MODULE 03 · LESSON 3.2

git init and git status

git init ٩git status

Learning objectives

- Create a folder and enter it from the terminal.
- Initialize a repository with `git init`.
- Read the output of `git status`.

Prerequisites

Lesson 3.1 and a configured Git (Lesson 2.3).

Introduction

Two commands begin every Git project: `git init` creates the repository, and `git status` tells you what Git sees. You will run `git status` hundreds of times.

Explanation

`mkdir my-first-project` makes a new folder. `cd my-first-project` moves into it. Choose a location such as Documents. Do not use a folder that already holds other projects.

`git init` turns the current folder into a repository. Git answers “Initialized empty Git repository in …/.git/”. Running it again is harmless: Git says it reinitialized.

`git status` shows the branch, whether there are commits, and which files are untracked, modified or staged.

In the output, **On branch main** is the current branch. **No commits yet** means the history is empty. **nothing to commit** means Git sees no files yet.

Real-world analogy

> `git init` is switching on the recorder. `git status` is asking the recorder what it currently sees.

Diagram

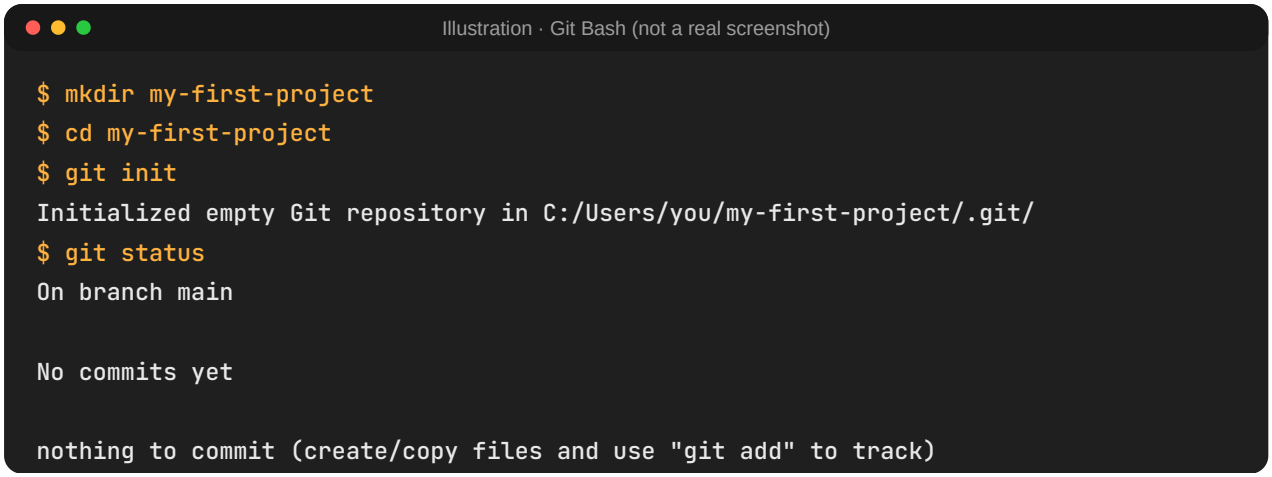


Figure 1 · Illustration of the expected output

Commands

`mkdir my-first-project`
`cd my-first-project`
`git init`
`git status`

Expected output: see Figure 1. The path in your output will be your own.

Common mistakes and solutions

"fatal: not a git repository"	You are outside a repository. Use `cd` to enter the project folder, or run `git init` first.
Running git init in the wrong folder.	Run `pwd` to see where you are before you initialize.
Typing the folder name with spaces.	Use names without spaces, such as `my-first-project`.

Practical exercise

> Create `my-first-project`, initialize it, and run `git status`.

Expected result

Git reports an empty repository on branch `main` with no commits yet.

Knowledge check

1. Which command creates a repository?

- ○ git new
- ○ git init
- ○ git start

2. What does “No commits yet” mean?

- ○ The history is empty
- ○ Git is broken
- ○ The repo is private

3. Which command shows what Git sees?

- ○ git look
- ○ git status
- ○ git show-all

Answers

- git init — git init initializes a repository.
- The history is empty — No commit has been made.
- git status — git status reports the working state.

Summary

You created a folder, ran git init to make it a repository, and read the result of git status.

Key vocabulary

git init	Create a repository in the current folder
git status	Show the state of files and branch
pwd	Print the current folder

Next lesson

Lesson 3.3 — Tracking your first files. Now you add real files and watch Git notice them.

MODULE 03 · LESSON 3.3

Tracking your first files

تتبّع ملفاتك الأولى

Learning objectives

- Create the three starter files.
- See untracked files in `git status`.
- Name the four file states.

Prerequisites

Lesson 3.2.

Introduction

Git sees your folder but does not track anything until you tell it to. Here you create the first files and watch how Git reports them.

Explanation

In VS Code choose **File → Open Folder** and open `my-first-project`. Create three files: `index.html`, `style.css` and `README.md`. Paste the contents below and **save each file**. A dot on a tab means it is not saved yet.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>My First Project</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, Git!</h1>
  <p>This is my first project.</p>
</body>
</html>
```

```
body {
  font-family: sans-serif;
  margin: 2rem;
}
```

```
h1 {
  color: #3B41E3;
}
```

```
# My First Project

A small website I use to learn Git and GitHub.
```

Run `git status`. Git lists the three files under **Untracked files**: it sees them but is not tracking them.

The four states: Untracked (new, unknown to Git), **Modified** (tracked file changed), **Staged** (chosen for the next commit), **Committed** (safely stored in history). In VS Code, untracked files show a **U** beside the name and modified files show an **M**. You will stage and commit them in Module 4.

Real-world analogy

> Untracked files are guests who walked into the house. Git sees them, but nobody has put them on the guest list yet.

Diagram

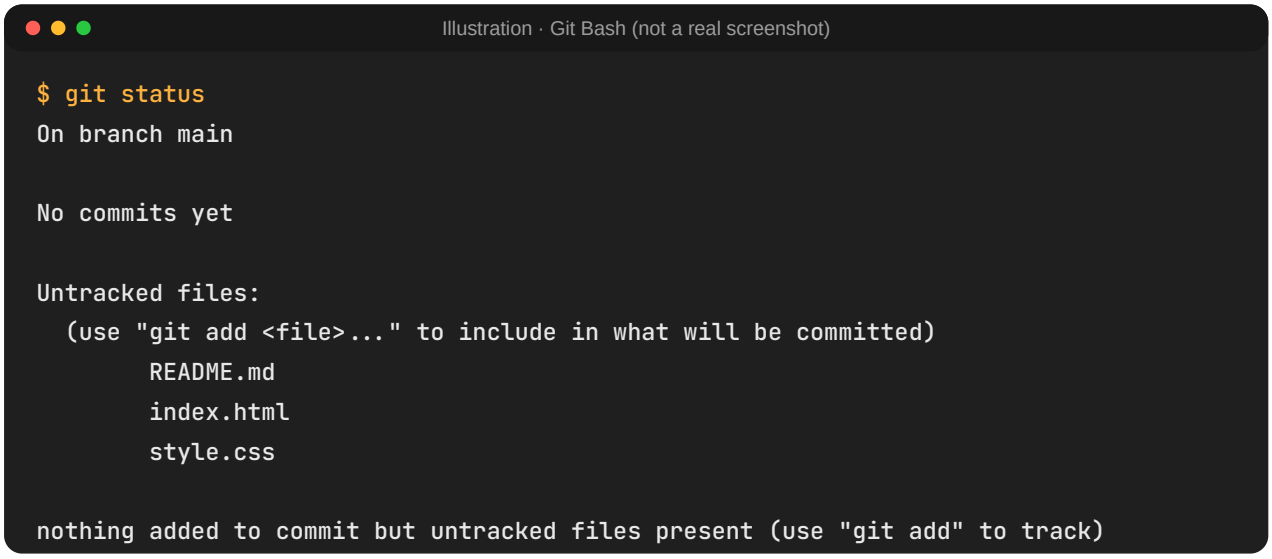


Figure 1 · Illustration of the expected output

Commands

`git status`

Run it after creating and saving the files. You can also copy the finished files from the course folder `starter-project/` .

Common mistakes and solutions

Files not saved.	Save with `Ctrl+S` . Git only sees saved changes.
Files created outside the project folder.	Check they are inside `my-first-project` .
Expecting git status to track the files.	It only reports. Tracking starts with git add.

Practical exercise

> Create the three files, save them, and run `git status` .

Expected result

All three files appear under “Untracked files”, as in Figure 1.

Knowledge check

1. What state is a brand-new file?

- ○ Pushed
- ○ Untracked
- ○ Committed

2. What does U mean in VS Code?

- ○ Untracked file
- ○ Updated online
- ○ Undo

3. Which state comes after staged?

- ○ Deleted
- ○ Committed
- ○ Untracked

Answers

- Untracked — Git has not seen it before.
- Untracked file — U marks an untracked file.
- Committed — Commit stores staged changes in history.

Summary

You created index.html, style.css and README.md, saw them as untracked, and learned the four file states.

Key vocabulary

Untracked	Not yet tracked by Git
Modified	A tracked file changed since the last commit
Staged	Marked for the next commit
Committed	Stored in the history

Next lesson

> **Module 4 — Git add, commit and history.** The most important module: you will save your first snapshots.

MODULE 03

Practical lab 3 · Create an HTML project and initialize Git

1. Create the folder `my-first-project` and enter it.
2. Run `git init`.
3. Open the folder in VS Code and create `index.html`, `style.css` and `README.md` (copy from `starter-project/`).
4. Save all files and run `git status`.

Check	Pass if…
Repository	`ls -a` shows `.git`
Files	Three files exist and are saved
Status	All three are listed as untracked

Evaluation: 3 of 3 checks = pass. Keep this folder: you will use it in every later module.

Module 3 quiz (pass mark: 70%)

1. Which command creates a repository?

- ○ git add
- ○ git push
- ○ git init

2. What does .git contain?

- ○ Your password
- ○ The repository history and settings
- ○ Your HTML files

3. What does git status show first?

- ○ The current branch
- ○ Your email
- ○ The weather

4. A new file shows as…

- ○ Merged
- ○ Untracked
- ○ Committed

5. Why save files before git status?

- ○ Windows requires it
- ○ To upload them
- ○ Git only sees saved changes

Answers

- git init — git init initializes a repository.
- The repository history and settings — It is the repository database.
- The current branch — The first line names the branch.
- Untracked — Git has not tracked it yet.
- Git only sees saved changes — Unsaved edits are not on disk.

04

MODULE 04

Git Add, Commit, and History

Choose what to save, save it with a clear message, and read the history you created.

MODULE 04 · LESSON 4.1

Understanding git add

فهم git add

Learning objectives

- Explain what staging means.
- Stage everything or only specific files.
- Tell `git add .` from `git add -A`.
- Inspect what is staged.
- Explain why secrets must never be committed.

Prerequisites

Module 3 and the `my-first-project` folder.

Introduction

Git does not save every change automatically. First you choose which changes belong in the next snapshot. That choice is called staging, and `git add` is the command for it.

Explanation

Staging means placing changes in the staging area, the waiting room for the next commit. `git add` never creates a commit. It only prepares one.

`git add index.html` stages one file. `git add index.html style.css` stages two. `git add .` stages every change in the current folder and its subfolders: new, modified and deleted files.

`git add -A` stages all changes in the whole repository, wherever you are inside it. From the project root, `.` and `-A` do the same thing. They differ when you run them inside a subfolder.

`git add -p` lets you stage piece by piece. Git shows each changed block (a *hunk*) and asks `Stage this hunk [y,n,q,a,d,s,e,?]?`. Answer `y` for yes, `n` for no, and `q` to quit.

Why stage selectively? If you fixed a bug and also started a new feature, two commits are clearer than one. Specific files or `-p` let you split the work.

Inspect. `git status` lists staged files under “Changes to be committed”. `git diff --staged` shows exactly what is staged. If you edit a file **after** staging it, that file

appears in both lists: the commit takes the version you staged, not your newer edit. Run `git add` again to include it.

Secrets. Never stage files with passwords, API keys or `.env` data. Once committed they stay in history. Check `git status` before `git add .`.

In VS Code. Open Source Control (`Ctrl+Shift+G`). The + next to a file stages it. The + next to **Changes** stages all of them (like `git add .`). The − unstages.

Real-world analogy

> Staging is choosing items for a photo. You arrange the objects you want in the frame first. The shutter (the commit) comes after.

Diagram

Illustration · Git Bash (not a real screenshot)

```
$ git add index.html
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        style.css
```

Figure 1 · Illustration of the expected output

Commands

`git add index.html`
`git add index.html style.css`
`git add .`
`git add -A`
`git add -p`
`git status`

Options: `.` current folder; `-A` all changes in the repo; `-p` choose hunks interactively. Figure 1 shows `git status` after staging one file.

Common mistakes and solutions

Thinking git add saves the work.	Nothing is saved until you commit.
Using git add . without looking.	Run `git status` first so no secret or junk file slips in.
Editing after staging and committing without adding again.	Run `git add` again, or the new edit stays out of the commit.

Practical exercise

> In `my-first-project`, stage only `index.html`. Run `git status`. Then run `git diff --staged`.

Expected result

`index.html` is under “Changes to be committed” and the other two files are still untracked, as in Figure 1.

Knowledge check

1. Does git add create a commit?

- ○ Only the first time
- ○ No, it only stages
- ○ Yes

2. Which command shows what is staged?

- ○ git diff –staged
- ○ git push
- ○ git init

3. Why not commit a .env file?

- ○ Git cannot read it
- ○ It may contain secrets that stay in history
- ○ It is too large

Answers

- No, it only stages — Staging prepares changes. Commit creates the snapshot.
- git diff –staged — It compares the staging area with the last commit.
- It may contain secrets that stay in history — Anything committed remains in history.

Summary

git add stages changes for the next commit. Stage specific files for clean commits, check with git status and git diff –staged, and never stage secrets.

Key vocabulary

Staging area	Waiting room for the next commit
git add -A	Stage all changes in the repository
Hunk	A changed block of a file

Next lesson

Lesson 4.2 — Understanding git commit. Your changes are staged. Now you save them.

MODULE 04 · LESSON 4.2

Understanding git commit

فهم git commit

Learning objectives

- Explain what a commit is.
- Create a commit with a good message.
- Read a commit hash.
- Tell good from bad commit messages.

Prerequisites

Lesson 4.1.

Introduction

A commit is the moment you save a snapshot into history. This is the heart of Git.

Explanation

A **commit** is a snapshot of the staged changes plus a message, an author and a date. It is stored in your local history and linked to the commit before it.

`git commit -m "Add homepage"` creates a commit with that message. `-m` means “message”. Keep the message inside quotes.

Git prints a line such as `[main (root-commit) 3f2a9c1] Add homepage`. **main** is the branch, **root-commit** marks the very first commit, and **3f2a9c1** is the start of the **commit hash**, a unique ID.

Commits stay on your computer. Nothing is on GitHub until you push (Module 5).

Good messages start with a verb and say what changed: `Add homepage`, `Fix navigation styling`, `Update README`. **Bad messages** say nothing: `changes`, `stuff`, `asdf`.

Meaningful commits. Each commit should be one logical change that works. That makes history readable and mistakes easy to undo.

Real-world analogy

> A commit is a save point in a video game with a label. Good labels let you find “before the boss fight” quickly.

Diagram

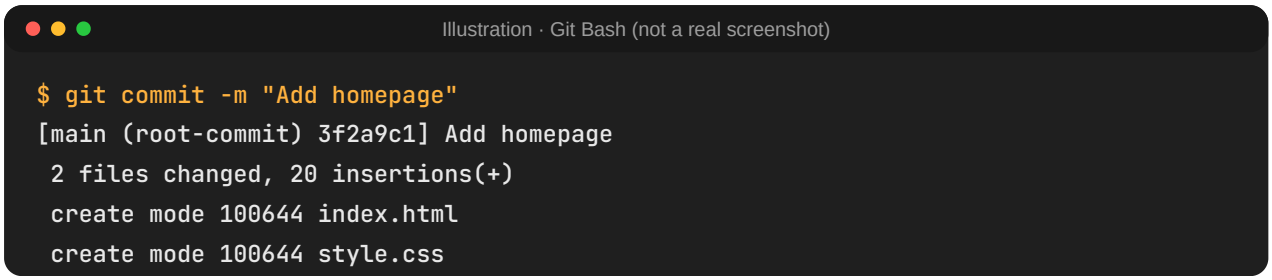


Figure 1 · Illustration of the expected output

Commands

```
git commit -m "Add homepage"
git commit -m "Fix navigation styling"
git commit -m "Update README"
```

Option: `-m` message. Run only after staging. Figure 1 shows the first commit of the project.

Common mistakes and solutions

“Author identity unknown”	Set `user.name` and `user.email` (Lesson 2.3).
“nothing to commit”	Nothing is staged. Run `git add` first.
Forgetting the quotes.	Put the message in double quotes.

Practical exercise

> Commit the staged `index.html` and `style.css` with the message `Add homepage`.

Expected result

Git prints a line starting `[main (root-commit)` and lists two created files, as in Figure 1.

Knowledge check

1. What does -m mean in git commit -m?

- ○ Move
- ○ Message
- ○ Merge

2. Which is a good message?

- ○ Add homepage
- ○ stuff
- ○ asdf

3. Where does a new commit live?

- ○ Nowhere
- ○ On your computer
- ○ On GitHub

Answers

- Message — It supplies the commit message.
- Add homepage — It starts with a verb and says what changed.
- On your computer — Push is the step that uploads it.

Summary

A commit saves staged changes with a message. Use clear messages, and remember commits are local until you push.

Key vocabulary

Commit hash	A unique ID for a commit
root-commit	The first commit in a repository
-m	Option to give the commit message

Next lesson

Lesson 4.3 — Reviewing Git history. You can now save snapshots. Next you will read them.

MODULE 04 · LESSON 4.3

Reviewing Git history

مراجعة سجل Git

Learning objectives

- Read the commit list with `git log`.
- Compare changes with `git diff` and `git diff --staged`.
- Inspect a commit with `git show`.

Prerequisites

Lesson 4.2.

Introduction

History is only useful if you can read it. These commands show what happened and when.

Explanation

`git log` lists commits, newest first. Each entry shows the full hash, author, date and message. If the list is long, Git opens a viewer: press **q** to leave it.

`git log --oneline` shows one short line per commit: a short hash and the message. `(HEAD → main)` marks where you are now.

`git diff` shows changes you made but have **not staged**. `git diff --staged` shows changes that are staged for the next commit. Lines starting with `+` were added and lines starting with `-` were removed.

`git show` displays the latest commit with its changes. `git show 3f2a9c1` shows a chosen commit.

`git status` remains your first check: it tells you what to do next.

Real-world analogy

> `git log` is the logbook of a ship. `git diff` compares today's page with what is already written.

Diagram

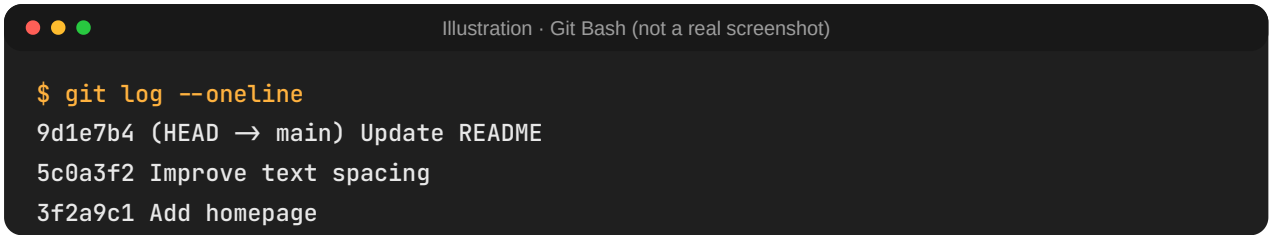


Figure 1 · Illustration of the expected output

Commands

`git log`
`git log --oneline`
`git diff`
`git diff --staged`
`git show`
`git status`

Figure 1 shows `git log --oneline` after three commits.

Common mistakes and solutions

Stuck inside git log.	Press `q` to quit the viewer.
Expecting git diff to show staged changes.	Use `git diff --staged` for those.
Copying a full hash by hand.	The first 7 characters are enough.

Practical exercise

> Edit `README.md` and run `git diff`. Stage it and run `git diff --staged`. Then run `git log --oneline`.

Expected result

The first diff shows your added lines with `+`. After staging, only `--staged` shows them. The log lists your commits.

Knowledge check

1. How do you leave the git log viewer?

- ○ Type exit()
- ○ Press q
- ○ Close Windows

2. Which command shows unstaged changes?

- ○ git diff
- ○ git log
- ○ git show

3. What does git log –oneline show?

- ○ Your passwords
- ○ One short line per commit
- ○ Only the last file

Answers

- Press q — q quits the pager.
- git diff — git diff compares your working files with the staging area.
- One short line per commit — A short hash and the message for each commit.

Summary

git log lists history, git diff and git diff –staged compare changes, and git show inspects one commit.

Key vocabulary

HEAD	The commit you are currently on
git diff	Show unstaged changes
git show	Show a commit and its changes

Next lesson

Module 5 — Connecting Git to GitHub. Your history is local. Next you send it online.

MODULE 04

Practical lab 4 · Create and inspect three commits

1. Run `git add index.html style.css`, then `git commit -m "Add homepage"`.
2. Add the line `p { line-height: 1.6; }` to `style.css`. Check with `git diff`, stage it, then `git commit -m "Improve text spacing"`.
3. Add one sentence to `README.md`. Run `git add README.md` and `git commit -m "Update README"`.
4. Run `git log --oneline` and `git show`.

Check	Pass if…
Three commits	`git log --oneline` lists three lines
Messages	Each starts with a verb and describes the change
Selective staging	Each commit contains only the intended files
Clean state	`git status` says “nothing to commit, working tree clean”

Evaluation: 4 of 4 checks = pass.

Module 4 quiz (pass mark: 70%)

1. Which command stages one file?

- ○ git push index.html
- ○ git log index.html
- ○ git add index.html

2. What does git add . do?

- ○ Uploads to GitHub
- ○ Stages changes in the current folder
- ○ Creates a commit

3. Which command saves staged changes?

- ○ git commit -m "message"
- ○ git add -p
- ○ git diff

4. How do you see staged changes?

- ○ git init
- ○ git diff –staged
- ○ git diff

5. What does (HEAD → main) mean in the log?

- ○ The file is deleted
- ○ The commit is on GitHub
- ○ You are on the latest commit of main

Answers

- git add index.html — git add followed by the file name stages it.
- Stages changes in the current folder — It stages; it does not commit or push.
- git commit -m "message" — commit creates the snapshot.
- git diff –staged — –staged compares the staging area with the last commit.
- You are on the latest commit of main — HEAD marks your current position.

05

MODULE 05

Connecting Git to GitHub

Link your local repository to GitHub and publish your first project online.

MODULE 05 · LESSON 5.1

Local vs remote repositories

المستودع المحلي مقابل البعيد

Learning objectives

- Explain what a GitHub repository is.
- Read a remote URL.
- Explain what `origin` means.
- Tell `main` from `origin/main`.
- Tell HTTPS from SSH URLs.

Prerequisites

Module 4 and your GitHub account.

Introduction

So far your history lives only on your computer. GitHub holds a second copy online. This lesson explains how the two copies relate before you connect them.

Explanation

A **GitHub repository** is the remote copy of your project. It stores the same commits you have locally, and adds a web page, sharing and teamwork tools.

A **remote** is a saved bookmark to another repository. Its **URL** says where it lives. The HTTPS form is `https://github.com/USER/REPO.git`. The SSH form is `git@github.com:USER/REPO.git`.

origin is only a name. By convention Git calls the main remote `origin`. You could name it differently, but everyone uses origin.

Branches. `main` is your local branch. `origin/main` is your computer's remembered copy of the remote's main branch (a *remote-tracking branch*). It updates only when you push, fetch or pull.

HTTPS or SSH? HTTPS is simpler for beginners: Git Credential Manager signs you in through the browser. SSH uses a key pair and no password prompts after setup. This course uses HTTPS.

Real-world analogy

> Your local repo is your notebook. GitHub is the photocopy kept at the library. `origin` is the library's name on your address list.

Diagram

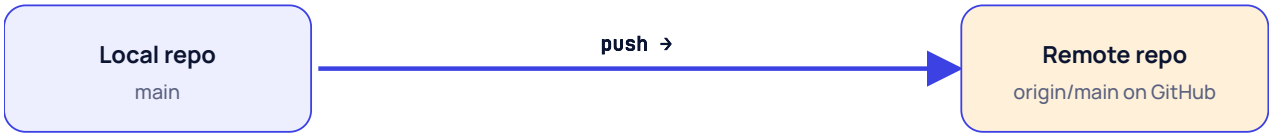


Figure 1 · Two copies of the same history

Commands

No new commands yet. Lesson 5.2 uses `git remote` and `git push`. Figure 1 shows how the two copies exchange commits.

Common mistakes and solutions

Thinking GitHub updates by itself.	GitHub only changes when you push.
Thinking origin is a special keyword.	It is a conventional name for a remote.
Copying the wrong URL type.	Use the HTTPS URL if you chose HTTPS.

Practical exercise

> Open your GitHub profile. Click New to start a repository and look at the options, but do not create it yet. Find the Public and Private choices.

Expected result

You can point to the repository name box and the visibility choice.

Knowledge check

1. What is a remote?

- ○ A commit message
- ○ A saved link to another repository
- ○ A deleted branch

2. What does origin mean?

- ○ A conventional name for the main remote
- ○ The first commit
- ○ Your username

3. What is origin/main?

- ○ A GitHub setting
- ○ Your computer's copy of the remote main branch
- ○ A new file

Answers

- A saved link to another repository — A remote stores the URL of another repository.
- A conventional name for the main remote — It is just a name.
- Your computer's copy of the remote main branch — It is a remote-tracking branch.

Summary

GitHub holds the remote copy. A remote is a named URL, usually origin. main is your branch; origin/main is the remembered remote one.

Key vocabulary

Remote	A named link to another repository
origin	Conventional name of the main remote
Remote-tracking branch	Local record of a remote branch, such as origin/main

Next lesson

Lesson 5.2 — git remote and git push. Now you connect and upload.

MODULE 05 · LESSON 5.2

git remote and git push

git remote ๑git push

Learning objectives

- Create an empty repository on GitHub.
- Connect it with `git remote add`.
- Push with `git push -u origin main`.
- Explain `-u` and upstream.
- Fix common push errors.

Prerequisites

Lesson 5.1 and at least one commit.

Introduction

This is the step where your work first appears online.

Explanation

Create the repository. On GitHub choose **New**. Name it `my-first-project`. Leave "Add a README" and other files **unchecked**, so the repository starts empty. Create it and copy the HTTPS URL.

`git remote add origin <URL>` saves the remote under the name origin. `git remote -v` lists remotes (Figure 1).

`git branch -M main` renames your current branch to `main`. It is safe, and ensures the name matches GitHub.

`git push -u origin main` uploads the `main` branch to `origin`. The **-u** option sets the **upstream**: it tells Git that local `main` goes with `origin/main`. After that, plain `git push` is enough.

First vs later pushes. The first push needs `-u origin main`. Later pushes are just `git push`.

Authentication. On the first push a Git Credential Manager window opens. Sign in to GitHub in the browser. Git remembers it afterwards.

Confirm. Refresh the repository page: your files and commits are there.

Real-world analogy

> `git remote add` saves the library's address. `git push` is posting the parcel. `-u` remembers the address so next time you just say "post it".

Diagram

Illustration · Git Bash (not a real screenshot)

```
$ git branch -M main
$ git push -u origin main
To https://github.com/YOUR-USERNAME/my-first-project.git
 * [new branch]      main → main
branch 'main' set up to track 'origin/main'.
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Figure 1 · Illustration of the expected output

Commands

`git remote add origin https://github.com/YOUR-USERNAME/my-first-project.git`
`git remote -v`
`git branch -M main`
`git push -u origin main`
`git push`

Options: `-v` verbose list; `-M` force-rename the branch; `-u` set upstream. Replace YOUR-USERNAME with yours.

Common mistakes and solutions

“remote origin already exists”	Use `git remote set-url origin <URL>` to change it.
“Repository not found”	Check the URL spelling and that you are signed in to the right account.
“Authentication failed”	Sign in again through the browser window. Do not use your password in the terminal.
“rejected … non-fast-forward”	The remote has commits you do not have. You will fix this with pull in Module 8.

Practical exercise

> Create the empty GitHub repository, add the remote, and push `main`.

Expected result

Git prints “branch ‘main’ set up to track ‘origin/main’” and GitHub shows your three commits and three files.

Knowledge check

1. What does -u do in git push -u?

- ○ Updates Git
- ○ Sets the upstream branch
- ○ Uploads faster

2. Which command lists remotes?

- ○ git remote -v
- ○ git push -v
- ○ git list

3. Do commits reach GitHub with git commit?

- ○ Only on Mondays
- ○ No, you must push
- ○ Yes, automatically

Answers

- Sets the upstream branch — It links local main with origin/main.
- git remote -v — git remote -v shows names and URLs.
- No, you must push — Commit is local; push uploads.

Summary

You created an empty GitHub repository, linked it as origin, and pushed main with -u. Later you only need git push.

Key vocabulary

Upstream	The remote branch your local branch tracks
git push	Upload commits to a remote
-u	Set the upstream while pushing

Next lesson

Lesson 5.3 — Exploring a GitHub repository. See what your push looks like online.

MODULE 05 · LESSON 5.3

Exploring a GitHub repository

استكشاف مستودع GitHub

Learning objectives

- Find files, commits and branches on GitHub.
- Read the README on the page.
- Name Issues and Pull requests.
- Find visibility and URLs.

Prerequisites

Lesson 5.2.

Introduction

Your pushed project now has a web page. Learn its parts, because you will use them every day.

Explanation

Code tab: files and folders, the branch selector (`main`), and the green **Code** button with the clone URL.

The **README** shows below the file list, formatted. It is the front page of the project.

Commits: click the clock icon with the commit count to see every commit with message, author and date.

Branches: lists all branches. You will create more in Module 7.

Issues track tasks and bugs. **Pull requests** propose changes for review (Module 9).

Settings controls visibility (Public or Private), collaborators, and later GitHub Pages. Change visibility carefully.

URLs. The page address is `https://github.com/USER/REPO` . The `.git` version of it is what Git uses to clone and push.

Real-world analogy

> The repository page is a shop window for your project: the front page (README), the stock list (files), the history book (commits).

Diagram

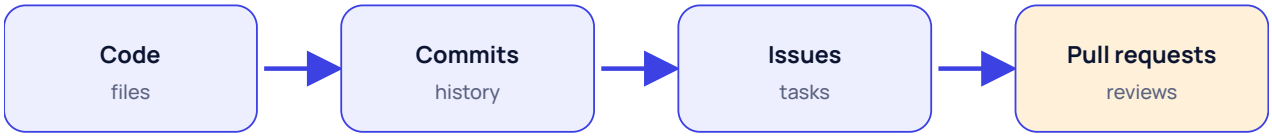


Figure 1 · Main tabs of a GitHub repository

Commands

No Git commands here. You work in the browser.

Common mistakes and solutions

Making a repo public by accident.	Check the visibility under Settings.
Editing files on GitHub then forgetting local.	Later you must pull those changes (Module 8).
Ignoring the README.	Write it early. It is the first thing people read.

Practical exercise

> Open your repository on GitHub. Find: the file list, the commit count, the branch selector, the README and the Settings tab.

Expected result

You can locate all five and open the commit list showing your three commits.

Knowledge check

1. Where do you see commit history?

- ○ Settings
- ○ The Commits link on the Code tab
- ○ The Issues tab

2. What are Pull requests for?

- ○ Proposing changes for review
- ○ Deleting repos
- ○ Installing Git

3. Where is visibility set?

- ○ README
- ○ Settings
- ○ Issues

Answers

- The Commits link on the Code tab — The commit count opens the full history.
- Proposing changes for review — They are how changes are reviewed and merged.
- Settings — Settings holds Public and Private.

Summary

A GitHub repository page shows code, commits, branches, README, issues, pull requests and settings, including visibility.

Key vocabulary

README	The project's front page
Issue	A tracked task or bug
Pull request	A proposal to merge changes

Next lesson

Module 6 — Git and GitHub in VS Code. You will do all of this with buttons.

MODULE 05

Practical lab 5 · Push your first project to GitHub

1. Create an empty repository named `my-first-project` on GitHub.
2. In your project folder run `git remote add origin <your URL>`.
3. Run `git remote -v`.
4. Run `git branch -M main` and `git push -u origin main`. Sign in when asked.
5. Refresh GitHub and open the commit list.

Check	Pass if…
Remote	`git remote -v` shows origin with your URL
Upload	GitHub shows your 3 files and 3 commits
Upstream	`git status` says the branch is up to date with `origin/main`

Evaluation: 3 of 3 checks = pass.

Module 5 quiz (pass mark: 70%)

1. Which command links a remote named origin?

- ○ git push origin
- ○ git clone origin
- ○ git remote add origin URL

2. What does git push -u origin main set?

- ○ Your email
- ○ The upstream branch
- ○ The commit message

3. What does origin usually mean?

- ○ The main remote
- ○ The first file
- ○ A branch

4. What does main versus origin/main show?

- ○ Two users
- ○ Local branch versus remembered remote branch
- ○ Two folders

5. Which URL type does this course use?

- ○ FTP
- ○ Bluetooth
- ○ HTTPS

Answers

- git remote add origin URL — remote add saves the URL under a name.
- The upstream branch — It tracks origin/main.
- The main remote — It is the conventional remote name.
- Local branch versus remembered remote branch — origin/main is a remote-tracking branch.
- HTTPS — HTTPS with Git Credential Manager.

06

MODULE 06

Git and GitHub in VS Code

Do everything you learned with buttons: stage, commit, push, branch and sync.

MODULE 06 · LESSON 6.1

VS Code Source Control

لوحة التحكم بالمصدر في VS Code

Learning objectives

- Open the Source Control view.
- Read the status letters beside files.
- Stage, stage all and discard changes.
- Write a commit message and commit.

Prerequisites

Module 4 and the project open in VS Code.

Introduction

VS Code has Git built in. The Source Control view gives you buttons for the commands you already know.

Explanation

Open it. Click the Source Control icon in the left bar (it looks like a branch) or press `Ctrl+Shift+G`. A number badge shows how many files changed.

Changed files. Files appear under **Changes**. A letter on the right shows the status: **M** modified, **U** untracked, **A** added (staged new file), **D** deleted. Click a file to open a side-by-side comparison with the last commit.

Stage Changes. Hover over a file and click **+**. This equals `git add file`. The file moves to **Staged Changes**.

Stage All Changes. Click **+** on the Changes heading. This equals `git add .`. Check the list first.

Unstage. Click **–** on a staged file. This equals `git restore --staged file`.

Discard Changes. The arrow-shaped button beside a file throws away your edits in that file. **Warning: this cannot be undone.** VS Code asks you to confirm.

Commit. Type a message in the box and click **Commit**. This equals `git commit -m "message"`. If nothing is staged, VS Code may offer to stage everything and

commit. Prefer to stage deliberately.

Real-world analogy

> Source Control is the dashboard of a car: the same engine (Git), but with buttons and gauges instead of typed instructions.

Diagram

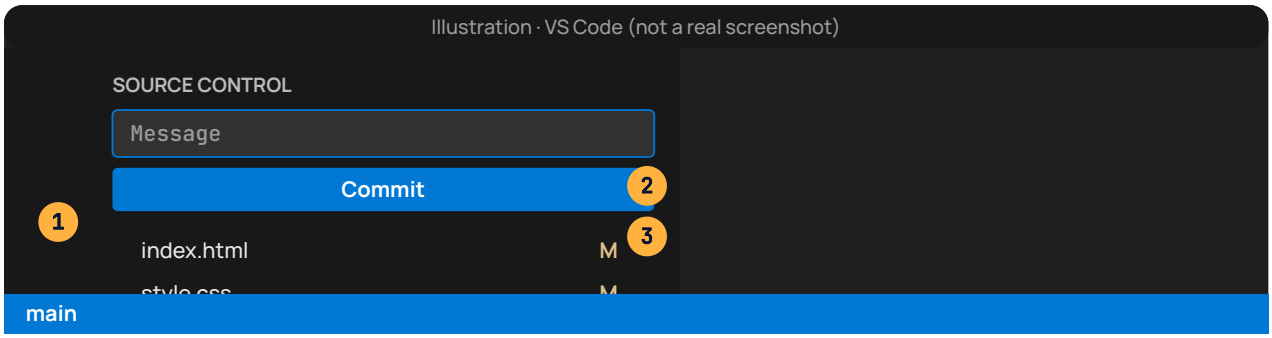


Figure 1 · (1) Source Control icon, (2) Stage a file, (3) Untracked file

Commands

Equivalents: **+** = `git add`; **−** = `git restore --staged`; **Commit** = `git commit -m`. Figure 1 shows the panel with three changed files.

Common mistakes and solutions

Clicking Discard by accident.	Discard deletes your edits. Read the confirmation carefully.
Committing with an empty message.	VS Code requires a message. Write a clear one.
Not seeing the Git view.	Open the project folder itself (File → Open Folder), not a parent folder.

Practical exercise

> Edit `index.html`, open Source Control, click the file to see the comparison, stage it with **+**, then unstage it with **−**.

Expected result

The file moves between Changes and Staged Changes, and the badge number updates.

Knowledge check

1. What does M mean beside a file?

- ○ Moved
- ○ Modified
- ○ Merged

2. Which button equals git add .?

- ○ + on the Changes heading
- ○ The Commit button
- ○ Discard

3. Can Discard Changes be undone?

- ○ Only on Fridays
- ○ No
- ○ Yes, always

Answers

- Modified — M marks a modified tracked file.
- + on the Changes heading — It stages every change.
- No — It permanently removes unsaved-to-Git edits.

Summary

Source Control shows changed files with status letters and gives buttons to stage, unstage, discard and commit.

Key vocabulary

Source Control	VS Code's built-in Git view
Staged Changes	Files ready for the next commit
Discard Changes	Delete your edits in a file permanently

Next lesson

> **Lesson 6.2 — Commit and push without the terminal.** A full workflow, step by step.

MODULE 06 · LESSON 6.2

Commit and push without the terminal

الحفظ والرفع بدون الطرفية

Learning objectives

- Complete edit → stage → commit → push in VS Code.
- Review differences before committing.
- Verify the result on GitHub.

Prerequisites

Lesson 6.1 and a project already pushed (Module 5).

Introduction

Now you do the whole daily workflow with the mouse.

Explanation

1. Open the project. File → Open Folder → `my-first-project`.

2. Edit `index.html`. Add `<p>Learning Git with Programming Worlds.</p>` under the heading. Save with `Ctrl+S`.

3. Open Source Control (`Ctrl+Shift+G`).

4. Review. Click `index.html`. The right side shows the new line in green.

5. Stage the file with **+**.

6. Write a message: `Add intro paragraph`.

7. Commit. Click **Commit**.

8. Push. A **Sync Changes** button appears (with arrows and a number). Click it. Sync pulls remote changes, then pushes yours.

9. Verify. Open your repository on GitHub and refresh. The new commit and the changed file are there.

If the button says **Publish Branch**, the branch has never been pushed. Click it once.

Real-world analogy

> It is the same parcel service as before, just booked online instead of by phone.

Diagram

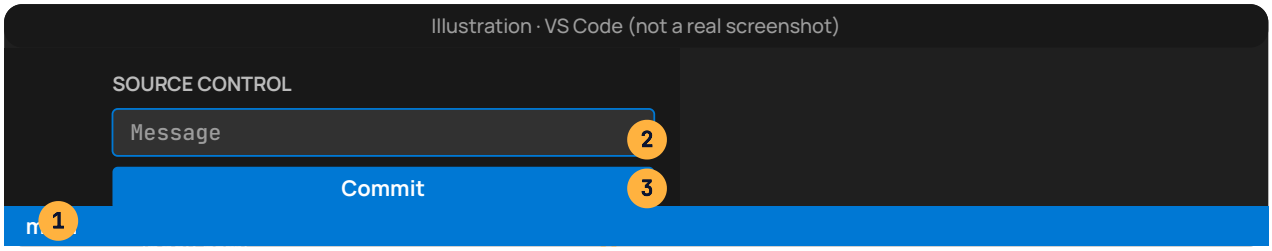


Figure 1 · (1) Source Control, (2) message box, (3) Commit

Commands

Terminal equivalent of steps 5 to 8: `git add index.html`, `git commit -m "Add intro paragraph"`, `git push`.

Common mistakes and solutions

Forgetting to save the file.	An unsaved file shows a dot and not in Source Control.
Sync asks to “pull and push”.	That is normal. Confirm.
Sign-in prompt appears.	Sign in to GitHub in the browser window and approve.

Practical exercise

> Do the nine steps with your own paragraph and message.

Expected result

GitHub shows the commit “Add intro paragraph” and the new line in `index.html`.

Knowledge check

1. Which VS Code button uploads your commit?

- ○ Stage All
- ○ Sync Changes
- ○ Discard

2. What does Publish Branch mean?

- ○ The branch is not on GitHub yet
- ○ The branch is deleted
- ○ The branch is merged

3. Where do you verify the push?

- ○ In Settings
- ○ On GitHub
- ○ In Notepad

Answers

- Sync Changes — Sync pulls then pushes.
- The branch is not on GitHub yet — It pushes a new branch.
- On GitHub — Refresh the repository page.

Summary

You edited, reviewed, staged, committed and synced from VS Code, then verified on GitHub.

Key vocabulary

Sync Changes	Pull then push
Publish Branch	Push a branch for the first time

Next lesson

Lesson 6.3 — Branches and history in VS Code.

MODULE 06 · LESSON 6.3

Branches and history in VS Code

الفروع والتاريخ في VS Code

Learning objectives

- Read the branch indicator.
- Create and switch branches.
- View file history.
- Know where conflicts and sign-in live.
- Tell built-in Git from GitLens.

Prerequisites

Lesson 6.2.

Introduction

Branches are covered in depth in Module 7. Here you only learn where the controls live.

Explanation

Branch indicator. The bottom-left of the window shows the current branch, such as `main`.

Switch or create. Click it. A list opens at the top: choose a branch to switch, or choose **Create new branch…** and type a name such as `feature/homepage`.

History. In the Explorer view, open the **Timeline** panel at the bottom to see the commits that changed the open file. Click one to compare. Recent versions also offer a graph of commits inside Source Control.

Compare changes. Click any changed file in Source Control to see old and new side by side.

Conflicts. If a merge conflicts, files appear under **Merge Changes**. Open one and use **Resolve in Merge Editor**. Module 7 teaches this.

Sync Changes pulls then pushes in one click.

GitHub sign-in. The Accounts icon at the bottom of the left bar shows who is signed in and lets you sign in or out.

GitLens is an optional extension that adds blame, richer history and more. It is not needed for this course. Everything above is built in.

Real-world analogy

> The status bar is a car's road sign: it always tells you which road (branch) you are on.

Diagram

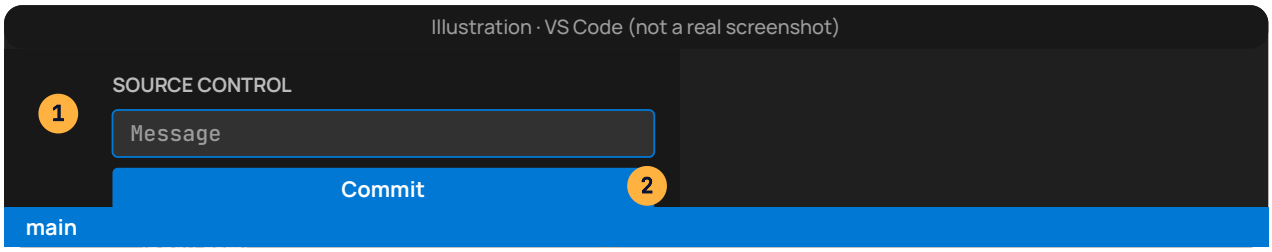


Figure 1 · (1) Branch indicator, (2) Changes

Commands

Equivalents: `git branch` shows the branch; `git switch -c feature/homepage` creates one; `git switch main` switches.

Common mistakes and solutions

Editing on main by accident.	Look at the bottom-left before you start.
Switch blocked by changes.	Commit your changes first.
Installing GitLens and getting lost.	Learn the built-in tools first.

Practical exercise

> Create a branch `feature/homepage` from the status bar, then switch back to `main`.

Expected result

The status bar shows `feature/homepage`, then `main` again.

Knowledge check

1. Where do you see the current branch?

- ○ Settings
- ○ Bottom-left status bar
- ○ The README

2. Is GitLens required?

- ○ No, it is optional
- ○ Yes
- ○ Only for pushing

3. Where do you sign in to GitHub?

- ○ The Timeline
- ○ The Accounts icon
- ○ The Terminal menu

Answers

- Bottom-left status bar — It is shown in the status bar.
- No, it is optional — Git is built into VS Code.
- The Accounts icon — Accounts manages sign-in.

Summary

Branch controls live in the status bar, history in Timeline, conflicts in the Merge Editor, and sign-in under Accounts.

Key vocabulary

Status bar	The bar at the bottom of VS Code
Timeline	Per-file history panel
GitLens	Optional extension for extra Git features

Next lesson

Module 7 — Branches and merging.

MODULE 06

Practical lab 6 · A full workflow using only VS Code

1. Open `my-first-project` in VS Code.
2. Add a paragraph to `index.html` and save.
3. Review the change in Source Control.
4. Stage only `index.html`, commit with `Add intro paragraph`, and click Sync Changes.
5. Check GitHub.

Check	Pass if…
No terminal	You used only buttons
Commit	Message starts with a verb
GitHub	New commit visible online

Evaluation: 3 of 3 = pass.

Module 6 quiz (pass mark: 70%)

1. Which icon opens Source Control?

- ○ The magnifier
- ○ The gear
- ○ The branch-shaped icon

2. What does U mean?

- ○ Undo
- ○ Untracked
- ○ Updated

3. What does Sync Changes do?

- ○ Pulls then pushes
- ○ Deletes files
- ○ Creates a repo

4. Where is the current branch shown?

- ○ Dashboard
- ○ Status bar
- ○ README

5. Is GitLens required?

- ○ Yes
- ○ Only on Windows
- ○ No

Answers

- The branch-shaped icon — Or press Ctrl+Shift+G.
- Untracked — New file Git does not track.
- Pulls then pushes — It synchronizes both ways.
- Status bar — Bottom-left.
- No — Optional extension.

07

MODULE 07

Git Branches and Merging

Work on ideas in parallel, combine them safely, and resolve conflicts calmly.

MODULE 07 · LESSON 7.1

Understanding branches

فهم الفروع

Learning objectives

- Define a branch.
- Explain why branches exist.
- Name feature and bug-fix branches.
- Explain how commits relate to branches.

Prerequisites

Module 6.

Introduction

Imagine trying a risky idea without touching your working project. Branches make that possible.

Explanation

A **branch** is a movable label that points to a commit. Making one costs almost nothing, because Git does not copy files.

main is the default branch. Keep it stable and working.

A **feature branch** holds new work, for example `feature/homepage`. A **bug-fix branch** holds a repair, for example `fix/nav-spacing`.

Parallel development. You can work on a feature while a teammate fixes a bug, each on a separate branch, without disturbing each other.

Commits and branches. When you commit, the current branch label moves forward to the new commit. **HEAD** tells Git which branch you are on. When ready, you **merge** the branch back into main (Lesson 7.2).

Real-world analogy

> A branch is a photocopy of a draft you scribble on. If the idea is good, you copy the changes into the original. If not, you throw the copy away.

Diagram

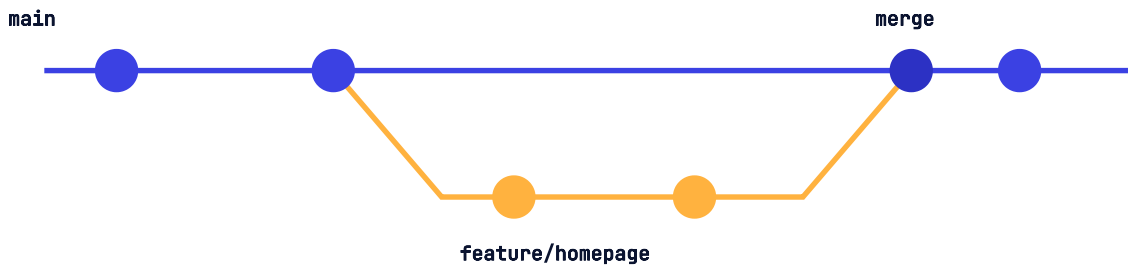


Figure 1 · A feature branch and a merge

Commands

No commands yet. Figure 1 shows a feature branch leaving main and returning through a merge.

Common mistakes and solutions

Thinking a branch is a full copy.	It is only a pointer to a commit, so it is fast and small.
Working on main directly.	Create a branch for each task.
Long-lived branches.	Merge often to avoid painful conflicts.

Practical exercise

> Draw on paper: main with three commits, a branch leaving after the second, two commits on it, then a merge.

Expected result

Your drawing matches Figure 1.

Knowledge check

1. What is a branch?

- ○ A GitHub account
- ○ A pointer to a commit
- ○ A copy of all files

2. What is a feature branch for?

- ○ New work
- ○ Deleting main
- ○ Installing Git

3. What does HEAD show?

- ○ Your email
- ○ The branch/commit you are on
- ○ The remote URL

Answers

- A pointer to a commit — Branches are lightweight pointers.
- New work — It isolates new work.
- The branch/commit you are on — HEAD is your current position.

Summary

Branches let you work in parallel safely. Commits move the current branch forward, and merges combine branches.

Key vocabulary

Branch	Movable pointer to a commit
Feature branch	Branch for new work
Merge	Combine branches

Next lesson

Lesson 7.2 — Creating and merging branches.

MODULE 07 · LESSON 7.2

Creating and merging branches

إنشاء الفروع ودمجها

Learning objectives

- List, create and switch branches.
- Merge a branch into main.
- Delete a merged branch.

Prerequisites

Lesson 7.1 and a clean working tree.

Introduction

Now you create a branch, do some work, and bring it back.

Explanation

`git branch` lists local branches. The star marks the current one.

`git switch -c feature/homepage` creates the branch and switches to it. `-c` means “create”.

Edit `index.html`, then `git add index.html` and `git commit -m "Add welcome section"`. The commit lands on the feature branch only.

`git switch main` returns to main. Your edit disappears from the folder because main does not have it yet.

`git merge feature/homepage` brings the commits into main. If main has not changed since you branched, Git just moves the main label forward: **Fast-forward**. If both changed, Git creates a merge commit.

`git branch -d feature/homepage` deletes the branch label after merging. `-d` refuses if it is not merged, which protects you.

To share the branch: `git push -u origin feature/homepage`.

Real-world analogy

> Switching branches is changing the film in a camera: the camera (your folder) stays, the picture it shows changes.

Diagram



Figure 1 · Illustration of the expected output

Commands

`git branch`
`git switch -c feature/homepage`
`git switch main`
`git merge feature/homepage`
`git branch -d feature/homepage`

Options: `-c` create; `-d` delete if merged. Figure 1 shows the merge and clean-up.

Common mistakes and solutions

"Your local changes would be overwritten"	Commit your changes before switching.
Merging into the wrong branch.	Switch to the branch that should receive the work first.
"not fully merged" on delete	Merge first, or confirm you really want `-D` (force).

Practical exercise

> Create `feature/homepage`, add a welcome paragraph, commit, switch to main, merge, and delete the branch.

Expected result

main contains the paragraph and `git branch` lists only main.

Knowledge check

1. Which command creates and switches?

- ○ git push name
- ○ git switch -c name
- ○ git merge name

2. What is Fast-forward?

- ○ Git moves the label ahead without a merge commit
- ○ A deleted file
- ○ A GitHub setting

3. Which branch receives the work in a merge?

- ○ Always the remote
- ○ The one you are on
- ○ The other one

Answers

- git switch -c name — -c creates the branch.
- Git moves the label ahead without a merge commit — Happens when main has not diverged.
- The one you are on — Merge into the current branch.

Summary

Create a branch with switch -c, commit on it, switch to main, merge, then delete the branch.

Key vocabulary

git switch	Change branches
Fast-forward	Merge by moving the label ahead
git branch -d	Delete a merged branch

Next lesson

> **Lesson 7.3 — Merge conflicts.**

MODULE 07 · LESSON 7.3

Merge conflicts

تعارضات الدمج

Learning objectives

- Explain why conflicts happen.
- Read conflict markers.
- Resolve with the VS Code Merge Editor.
- Finish the merge.

Prerequisites

Lesson 7.2.

Introduction

A conflict is not an error. It is Git asking you to decide.

Explanation

Why. Two branches changed the same lines differently. Git cannot guess which to keep.

Identify. The merge stops with `CONFLICT (content): Merge conflict in index.html`. `git status` lists the file under “Unmerged paths” as “both modified”.

Markers. Git writes both versions into the file. Between `<<<<<<< HEAD` and `=======` is your current branch. Between `=======` and `>>>>>>> feature/homepage` is the incoming branch (Figure 1).

Resolve. In VS Code the file shows buttons above the conflict: **Accept Current**, **Accept Incoming**, **Accept Both**, or open **Resolve in Merge Editor** for a three-pane view. Choose or combine, and make sure no markers remain.

Test. Open the page and check it still works.

Finish. Run `git add index.html` then `git commit` (or `git merge --continue`). Git proposes a merge message.

Escape hatch. `git merge --abort` cancels the merge and returns to the state before it.

Real-world analogy

> Two editors rewrote the same sentence. The author must read both and write the final sentence.

Diagram

Illustration · Git Bash (not a real screenshot)

```
$ git merge feature/homepage
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

<<<<<<< HEAD
<h1>Hello, World!</h1>
=======
<h1>Welcome to my site</h1>
>>>>>>> feature/homepage
```

Figure 1 · Illustration: a file with conflict markers

Commands

`git merge feature/homepage`
`git status`
`git add index.html`
`git commit`
`git merge --abort`

Figure 1 shows the conflict markers in the file.

Common mistakes and solutions

Leaving the markers in the file.	Search for `<<<<<<<` before committing.
Panicking.	Use `git merge --abort` and try again calmly.
Not testing after resolving.	Always open the result.

Practical exercise

> Read the lab in this module and resolve the conflict it creates.

Expected result

The file has one heading, no markers, and `git status` is clean after the merge commit.

Knowledge check

1. Why do conflicts happen?

- ○ GitHub is offline
- ○ Two branches changed the same lines
- ○ Git is broken

2. What does HEAD mean between the markers?

- ○ Your current branch version
- ○ The remote
- ○ The deleted version

3. How do you cancel a conflicted merge?

- ○ git push
- ○ git merge –abort
- ○ git delete

Answers

- Two branches changed the same lines — Git cannot choose automatically.
- Your current branch version — The first block is your side.
- git merge –abort — It restores the pre-merge state.

Summary

Conflicts mean Git needs your decision. Edit the file, remove markers, test, add, and commit, or abort.

Key vocabulary

Conflict	Same lines changed differently
Conflict markers	Lines with `<<<<<<<`, `=======`, `>>>>>>>`
Merge Editor	VS Code three-pane resolver

Next lesson

Module 8 — Clone, fetch and pull.

MODULE 07

Practical lab 7 · Create, merge and resolve a conflict

1. Make sure `git status` is clean on main.
2. `git switch -c feature/homepage`. Change the heading line to `<h1>Welcome to my site</h1>`. Commit: `Update heading on feature branch`.
3. `git switch main`. Change the same line to `<h1>Hello, World!</h1>`. Commit: `Update heading on main`.
4. Run `git merge feature/homepage`. A conflict appears.
5. Open `index.html`, keep `<h1>Welcome to my site</h1>` and remove all markers.
6. `git add index.html`, then `git commit`. Delete the branch with `git branch -d feature/homepage`.

Check	Pass if…
Conflict	You saw CONFLICT in index.html
Resolution	No markers remain
History	`git log --oneline` shows the merge commit

Evaluation: 3 of 3 = pass.

Module 7 quiz (pass mark: 70%)

1. Which command lists branches?

- ○ git log
- ○ git diff
- ○ git branch

2. How do you create and switch?

- ○ git add name
- ○ git switch -c name
- ○ git merge name

3. What does the merge do?

- ○ Combines another branch into the current one
- ○ Deletes a branch
- ○ Uploads to GitHub

4. What are `<<<<<<<` markers?

- ○ Passwords
- ○ Conflict markers
- ○ Comments

5. Which command cancels a merge?

- ○ git reset
- ○ git push
- ○ git merge –abort

Answers

- git branch — The star marks the current one.
- git switch -c name — -c means create.
- Combines another branch into the current one — You merge into the current branch.
- Conflict markers — They mark the two versions.
- git merge –abort — It restores the state before the merge.

08

MODULE 08

Clone, Fetch and Pull

Get a copy of a repository and keep it in sync with GitHub.

MODULE 08 · LESSON 8.1

Cloning a repository

نسخ مستودع

Learning objectives

- Explain what clone does.
- Clone with HTTPS.
- Choose a folder.
- Verify with git remote -v.

Prerequisites

Module 5 and a GitHub repository.

Introduction

Cloning gives you a full local copy of a repository, including its whole history.

Explanation

`git clone` downloads the repository, creates a folder, and sets up the remote named **origin** for you. You do not need `git init` or `git remote add`.

On GitHub, open the repository, click the green **Code** button, choose **HTTPS**, and copy the URL.

Open Git Bash in the folder where you want the project, for example `Documents`.

Run `git clone URL`. Git creates a folder with the repository name.

`cd my-first-project`, then `git remote -v` to see origin. `git log --oneline` shows the history you received.

In VS Code: **Source Control** → **Clone Repository**, paste the URL, choose a parent folder, and click **Open** when asked.

A private repository asks you to sign in. Git Credential Manager opens a browser window for that.

Real-world analogy

> Cloning is photocopying an entire notebook, including every page ever written, and getting a pointer back to the original.

Diagram

Illustration · Git Bash (not a real screenshot)

```
$ git clone https://github.com/YOUR-USERNAME/my-first-project.git
Cloning into 'my-first-project'...
remote: Enumerating objects: 9, done.
Receiving objects: 100% (9/9), done.
$ cd my-first-project
$ git remote -v
origin  https://github.com/YOUR-USERNAME/my-first-project.git (fetch)
origin  https://github.com/YOUR-USERNAME/my-first-project.git (push)
```

Figure 1 · Illustration: cloning and checking the remote

Commands

```
git clone https://github.com/YOUR-USERNAME/my-first-project.git
cd my-first-project
git remote -v
```

Figure 1 shows the flow. Do not clone inside another repository.

Common mistakes and solutions

Cloning inside a folder that is already a repository.	Pick a normal folder such as Documents.
“Repository not found”	Check the URL and that you are signed in to the right account.
Using the wrong folder name afterwards.	Run `cd` into the cloned folder before other commands.

Practical exercise

> Clone your repository into a new folder next to the original and compare the two.

Expected result

The clone has the same files and `git log --oneline` shows the same commits.

Knowledge check

1. What does clone create?

- ○ A new GitHub account
- ○ A local copy with history and origin
- ○ Only one file

2. Which URL type does this course use?

- ○ HTTPS
- ○ FTP
- ○ Bluetooth

3. Do you need git init after clone?

- ○ Only on Windows
- ○ No
- ○ Yes

Answers

- A local copy with history and origin — It copies everything.
- HTTPS — HTTPS with Credential Manager.
- No — Clone already creates the repository.

Summary

Clone copies a repository, with history, and sets origin for you.

Key vocabulary

clone	Copy a remote repository locally
origin	Default remote name
HTTPS URL	Web address of the repository

Next lesson

Lesson 8.2 — Fetch.

MODULE 08 · LESSON 8.2

Fetching changes

جلب التغييرات

Learning objectives

- Explain fetch.
- Compare local and remote.
- Read “behind” and “ahead”.

Prerequisites

Lesson 8.1.

Introduction

Fetch downloads new work from GitHub without changing your files. It is the safe way to look first.

Explanation

Someone edited the README on GitHub. Your computer does not know yet.

`git fetch` updates your **remote-tracking branches** such as `origin/main`. Your own `main` and your files stay untouched.

`git status` now says your branch is **behind** origin/main by one commit.

Inspect with `git log --oneline main..origin/main` (what you do not have) or `git diff main origin/main`.

“Ahead” means you have commits to push. “Behind” means GitHub has commits to pull. Both at once is “diverged”.

In VS Code, the status bar shows arrows with counts. The Source Control menu (…) has a **Fetch** command.

Real-world analogy

> Fetch is checking the mailbox and reading the letters at the door without bringing them into your room.

Diagram

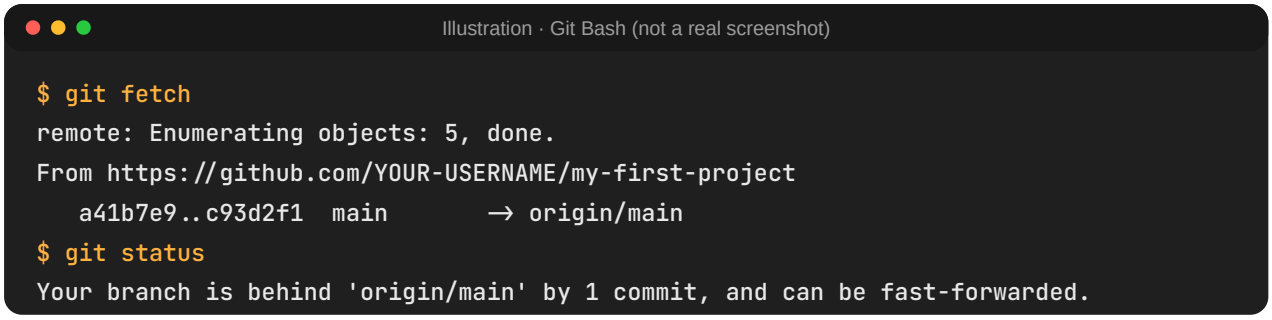


Figure 1 · Illustration: fetch then status

Commands

`git fetch`
`git status`
`git log --oneline main..origin/main`

Figure 1 shows the output after a fetch.

Common mistakes and solutions

Thinking fetch changes your files.	It does not. It only updates origin/*.
Forgetting to fetch before comparing.	Status uses your last known copy of origin.
Panic at “diverged”.	Pull and resolve (Lesson 8.3).

Practical exercise

> Edit README.md on GitHub (the pencil icon), commit there, then fetch locally.

Expected result

`git status` says you are behind by 1 commit and your README is unchanged.

Knowledge check

1. Does fetch change your working files?

- ○ Only on Linux
- ○ No
- ○ Yes, always

2. What does “behind by 1” mean?

- ○ GitHub has one commit you lack
- ○ You have one extra commit
- ○ Git is broken

3. Which branch is updated by fetch?

- ○ feature
- ○ origin/main
- ○ main

Answers

- No — It only updates remote-tracking branches.
- GitHub has one commit you lack — You need to pull.
- origin/main — The remote-tracking branch.

Summary

Fetch looks at GitHub safely. Status then tells you if you are behind or ahead.

Key vocabulary

fetch	Download remote changes without merging
behind	Remote has commits you lack
ahead	You have commits to push

Next lesson

Lesson 8.3 — Pull.

MODULE 08 · LESSON 8.3

Pulling changes

سحب التغييرات

Learning objectives

- Explain pull = fetch + merge.
- Pull safely.
- Handle a pull conflict.

Prerequisites

Lesson 8.2 and Module 7.

Introduction

Pull brings GitHub changes into your branch and files in one step.

Explanation

`git pull` runs `git fetch` and then merges `origin/main` into your current branch.

Commit or stash your work first. A clean tree makes pulls simple.

Run `git pull`. If you are only behind, you get a fast-forward (Figure 1).

If you are ahead too, Git creates a merge commit and may open an editor for the message. Save and close it.

Conflicts are handled exactly as in Lesson 7.3: edit, `git add`, `git commit`.

Good routine: **pull before you start working** and before you push.

In VS Code: **Pull** in the Source Control menu, or **Sync Changes** (pull then push).

Real-world analogy

> Pull is fetching the letters and then filing them into your folder.

Diagram



Figure 1 · Illustration: a fast-forward pull

Commands

`git pull`
`git log --oneline`

Figure 1 shows a fast-forward pull.

Common mistakes and solutions

Pulling with uncommitted edits to the same file.	Commit first.
“fatal: refusing to merge unrelated histories”	You mixed two separate repositories. Re-clone instead of forcing.
Not pulling before push.	Push is rejected; pull first.

Practical exercise

> Pull the README change you made on GitHub.

Expected result

Your README.md now has the new text and the log shows the GitHub commit.

Knowledge check

1. git pull equals…

- ○ add + commit
- ○ fetch + merge
- ○ delete + clone

2. When should you pull?

- ○ Before starting work and before push
- ○ Never
- ○ Only on Fridays

3. What if the pull conflicts?

- ○ Reinstall Git
- ○ Resolve as in a merge
- ○ Delete the repo

Answers

- fetch + merge — It does both steps.
- Before starting work and before push — It avoids rejected pushes.
- Resolve as in a merge — Same process.

Summary

Pull = fetch + merge. Pull often, keep your tree clean.

Key vocabulary

pull	Fetch and merge
Sync Changes	VS Code pull then push
diverged	Both sides have new commits

Next lesson

Module 9 — Collaboration and pull requests.

MODULE 08

Practical lab 8 · Clone, fetch and pull

1. Clone your repository into a new folder: `git clone URL my-first-project-copy` .
2. Open the original on GitHub, edit README.md with the pencil icon, and commit with the message `Update README on GitHub` .
3. In the copy folder run `git fetch` then `git status` . Confirm “behind by 1”.
4. Run `git pull` .
5. Run `git log --oneline` .

Check	Pass if…
Clone	Copy has the same files
Fetch	Status says behind by 1
Pull	Log shows the GitHub commit

> **Evaluation:** 3 of 3 = pass.

Module 8 quiz (pass mark: 70%)

1. What does git clone do?

- ○ Deletes a repo
- ○ Creates an account
- ○ Copies a repository locally

2. Does git fetch change your files?

- ○ Only README
- ○ No
- ○ Yes

3. git pull is…

- ○ fetch + merge
- ○ add + commit
- ○ push + clone

4. “behind by 1” means…

- ○ Nothing
- ○ Remote has a commit you lack
- ○ You must push

5. Which remote name does clone set?

- ○ server
- ○ main
- ○ origin

Answers

- Copies a repository locally — It downloads everything.
- No — It updates origin/*.
- fetch + merge — Two steps in one.
- Remote has a commit you lack — Pull to catch up.
- origin — origin is the default.

09

MODULE 09

Collaboration and Pull Requests

Propose changes on a branch, get them reviewed, and merge them on GitHub.

MODULE 09 · LESSON 9.1

The branch-and-push workflow

سير العمل بالفروع

Learning objectives

- Describe the team workflow.
- Create and push a feature branch.
- Know collaborators versus forks.

Prerequisites

Modules 7 and 8.

Introduction

Teams never commit straight to main. They work on branches and propose changes.

Explanation

Workflow. Update main, create a branch, commit, push the branch, open a pull request, get a review, merge, then clean up (Figure 1).

Start every task with `git switch main` and `git pull` so your branch starts from the latest work.

Create the branch: `git switch -c feature/about-page`. Use short, clear names: `feature/…`, `fix/…`, `docs/…`.

Commit small, focused changes with clear messages.

Publish: `git push -u origin feature/about-page`. GitHub then shows a yellow banner with **Compare & pull request**.

Collaborator. The owner invites you under *Settings → Collaborators*. You can push branches to the same repository.

Fork. If you are not a collaborator, you click **Fork** to make your own copy, push there, and open a pull request to the original. Forks are how open-source contributions work.

Real-world analogy

> You write your chapter in a separate document and hand it to the editor. The editor decides when it joins the book.

Diagram

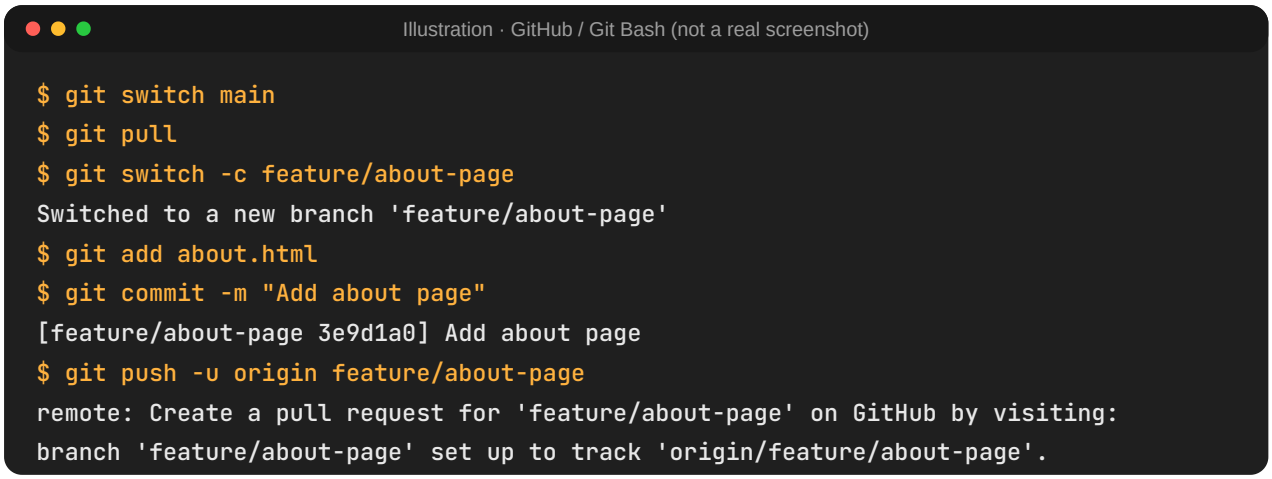


Figure 1 · Illustration: branch, commit, push

Commands

`git switch main`

`git pull`

`git switch -c feature/about-page`

`git push -u origin feature/about-page`

Figure 1 shows these commands in order.

Common mistakes and solutions

Branching from an old main.	Pull main first.
Huge branches with many unrelated changes.	One purpose per branch.
Pushing to main on a protected repository.	Push a branch instead.

Practical exercise

> Create `feature/about-page`, add `about.html`, commit, and push it.

Expected result

GitHub lists the branch and offers Compare & pull request.

Knowledge check

1. Where do teams commit new work?

- ○ Nowhere
- ○ On a branch
- ○ Directly on main

2. What does -u do in the push?

- ○ Sets the upstream
- ○ Deletes
- ○ Uploads twice

3. When do you fork?

- ○ Never
- ○ You cannot push to the original
- ○ Always

Answers

- On a branch — Branches protect main.
- Sets the upstream — Later you can just git push.
- You cannot push to the original — Contributors use forks.

Summary

Pull main, branch, commit, push the branch. Forks are for outside contributors.

Key vocabulary

Feature branch	Branch for one change
Fork	Your own GitHub copy of someone else's repository
Collaborator	Person with write access

Next lesson

Lesson 9.2 — Opening a pull request.

MODULE 09 · LESSON 9.2

Opening a pull request

فتح طلب دمج

Learning objectives

- Define a pull request.
- Open one on GitHub.
- Write a good title and description.

Prerequisites

Lesson 9.1.

Introduction

A pull request (PR) asks the team to review and merge your branch into another branch.

Explanation

On GitHub, click **Compare & pull request** (or the **Pull requests** tab → **New pull request**).

Check the two branches at the top: **base** is where changes go (main) and **compare** is your branch.

Write a clear **title**, for example `Add about page`.

Write the **description**: what changed, why, and how to check it. Mention an issue with `Closes #3` if there is one.

Look at the **Files changed** tab. Review your own diff before asking others.

Click **Create pull request**. Optionally choose reviewers on the right. Use **Create draft pull request** if the work is not ready.

Any new commit you push to the same branch appears in the PR automatically.

Real-world analogy

> A PR is a cover letter attached to your chapter: it explains what you changed and invites feedback.

Diagram

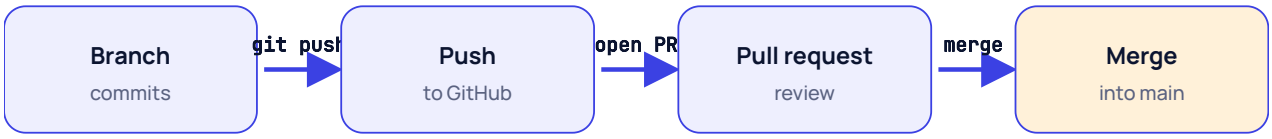


Figure 1 · The pull request flow

Commands

No new Git command is needed. A PR lives on GitHub; you push commits as usual.

Figure 1 summarises the pull request flow.

Common mistakes and solutions

Base set to the wrong branch.	Check base and compare before creating.
Empty description.	Always explain what and why.
Opening a PR with unrelated changes.	Split into separate PRs.

Practical exercise

> Open a pull request from `feature/about-page` into main with a title and description.

Expected result

The PR page shows your commit and “Able to merge”.

Knowledge check

1. What is a pull request?

- ○ A Git command
- ○ A request to review and merge a branch
- ○ A download

2. Which is the base branch?

- ○ Where changes will go
- ○ Your branch
- ○ The remote

3. A draft PR means…

- ○ Deleted
- ○ Not ready for review
- ○ Already merged

Answers

- A request to review and merge a branch — PRs live on GitHub.
- Where changes will go — Usually main.
- Not ready for review — It signals work in progress.

Summary

A PR proposes a branch for review. Give it a clear title and description.

Key vocabulary

Pull request	Proposal to merge a branch
base	Target branch
compare	Your branch

Next lesson

Lesson 9.3 — Review and merge.

MODULE 09 · LESSON 9.3

Review, merge and clean up

المراجعة والدمج والتنظيف

Learning objectives

- Review a PR.
- Merge on GitHub.
- Update local main and delete the branch.

Prerequisites

Lesson 9.2.

Introduction

A review makes the code better, and merging makes it official.

Explanation

Reviewers open **Files changed**, hover a line, click the blue **+**, and write a comment. **Review changes** lets them **Comment**, **Approve** or **Request changes**.

To answer feedback, commit to the same branch and push. The PR updates itself.

When approved and checks pass, click **Merge pull request** then **Confirm merge**. Other options are **Squash and merge** (one commit) and **Rebase and merge**. This course uses the default merge.

GitHub shows **Delete branch**. Click it; the work is safe in main.

Locally: `git switch main`, `git pull` (Figure 1), then `git branch -d feature/about-page`.

If the PR shows conflicts, pull main into your branch (`git pull origin main` while on your branch), resolve them as in Lesson 7.3, and push.

Be kind in reviews: comment on the work, suggest, and explain.

Real-world analogy

> The editor reads your chapter, leaves notes, and finally approves it into the book. You then update your own copy of the book.

Diagram



Figure 1 · Illustration: cleaning up after a merge

Commands

`git switch main`
`git pull`
`git branch -d feature/about-page`

Optional: `git push origin --delete feature/about-page` removes the remote branch if GitHub did not.

Common mistakes and solutions

Forgetting to pull after merging on GitHub.	Your local main is still old until you pull.
Merging your own PR without review in a team.	Follow team rules.
Deleting a branch before it is merged.	Merge first.

Practical exercise

> Merge your PR, delete the branch on GitHub, pull main locally, and delete the local branch.

Expected result

about.html exists on main locally and `git branch` shows only main.

Knowledge check

1. Who clicks Merge pull request?

- ○ Git automatically
- ○ Someone with permission after approval
- ○ Anyone anonymously

2. After merging on GitHub you should…

- ○ Pull main locally
- ○ Reinstall
- ○ Fork again

3. Request changes means…

- ○ The repo is deleted
- ○ Fixes are needed before merge
- ○ The PR is merged

Answers

- Someone with permission after approval — Teams define the rules.
- Pull main locally — Sync your local copy.
- Fixes are needed before merge — Update and push.

Summary

Review, approve, merge, delete the branch, then pull main.

Key vocabulary

Approve	Reviewer accepts the PR
Squash and merge	Merge as a single commit
Request changes	Reviewer asks for fixes

Next lesson

> **Module 10 — Undoing mistakes.**

MODULE 09

Practical lab 9 · Your first pull request

1. `git switch main` and `git pull`.
2. `git switch -c feature/about-page`.
3. Create `about.html` with a heading and a paragraph. Commit: `Add about page`.
4. `git push -u origin feature/about-page`.
5. On GitHub, click **Compare & pull request**, write a title and description, and create the PR.
6. Review **Files changed**, then **Merge pull request**, then **Delete branch**.
7. Locally: `git switch main`, `git pull`, `git branch -d feature/about-page`.

Check	Pass if…
PR	PR page exists with a description
Merge	about.html is on main on GitHub
Cleanup	Only main remains locally

Evaluation: 3 of 3 = pass.

Module 9 quiz (pass mark: 70%)

1. Where do you do new work?

- ○ On main
- ○ Offline only
- ○ On a branch

2. What is a PR?

- ○ A clone
- ○ A request to review and merge
- ○ A download

3. What do you do after merging on GitHub?

- ○ Pull main locally
- ○ Fork
- ○ Nothing

4. What does Approve mean?

- ○ Push failed
- ○ Reviewer accepts the changes
- ○ Branch deleted

5. Why write a description?

- ○ It is required by Git
- ○ For fun
- ○ So reviewers understand what and why

Answers

- On a branch — Branches protect main.
- A request to review and merge — It lives on GitHub.
- Pull main locally — Sync your computer.
- Reviewer accepts the changes — It allows merge.
- So reviewers understand what and why — Context speeds reviews.

10

MODULE 10

Undoing Mistakes Safely

Restore files, fix commits, and recover from errors without panic.

MODULE 10 · LESSON 10.1

Undo changes before you commit

التراجع قبل الـ commit

Learning objectives

- Discard a file edit.
- Unstage a file.
- Save work aside with stash.

Prerequisites

Module 4.

Introduction

Most mistakes happen before a commit, and they are the easiest to fix.

Explanation

`git restore index.html` throws away your edits in that file and returns it to the last commit. **This cannot be undone**, because Git never saved those edits.

`git restore --staged style.css` removes a file from the staging area but keeps your edits. It is the opposite of `git add`.

Always run `git status` first so you know which files are staged and which are not.

`git stash push -m "work in progress"` puts your uncommitted edits in a safe drawer and gives you a clean tree. `git stash list` shows drawers and `git stash pop` brings the latest back.

Use stash when you need to switch branches or pull but are not ready to commit.

In VS Code: hover a file in Source Control; the ↶ **Discard Changes** button restores it, and the **–** button unstages. Discard asks for confirmation, and it is also irreversible.

Real-world analogy

> Restore is erasing a pencil line back to the last printed page. Stash is sliding your messy desk into a drawer for later.

Diagram



Figure 1 · Illustration: restore and unstage

Commands

`git restore FILE`
`git restore --staged FILE`
`git stash push -m "msg"`
`git stash pop`

Figure 1 shows restore and unstage.

Common mistakes and solutions

Using restore on work you want to keep.	Stash or commit first.
Confusing restore with restore –staged.	The first discards edits; the second only unstages.
Forgetting stashed work.	Check `git stash list`.

Practical exercise

> Edit index.html, discard the edit. Then stage style.css and unstage it.

Expected result

`git status` shows index.html clean and style.css modified but not staged.

Knowledge check

1. What does git restore FILE do?

- ○ Pushes
- ○ Discards your edits
- ○ Commits

2. What does restore –staged do?

- ○ Unstages but keeps edits
- ○ Deletes the file
- ○ Pushes

3. git stash is for…

- ○ Cloning
- ○ Setting work aside
- ○ Deleting history

Answers

- Discards your edits — It is irreversible.
- Unstages but keeps edits — It undoes git add.
- Setting work aside — Pop brings it back.

Summary

Restore discards edits, restore –staged unstages, and stash parks work.

Key vocabulary

restore	Bring a file back to a saved state
unstage	Remove from staging area
stash	Temporary storage for edits

Next lesson

Lesson 10.2 — Fix a commit.

MODULE 10 · LESSON 10.2

Fix or revert a commit

تعديل أو عكسه commit

Learning objectives

- Amend the last commit.
- Revert a commit.
- Choose between amend and revert.

Prerequisites

Lesson 10.1.

Introduction

Sometimes the mistake is already in a commit. The right tool depends on whether you pushed it.

Explanation

Not pushed yet. `git commit --amend -m "Better message"` replaces the last commit. To add a forgotten file, `git add file` first, then amend.

Amend **rewrites** the commit (it gets a new ID). Never amend a commit others may already have.

Already pushed or shared. `git revert HEAD` creates a **new commit** that cancels the old one. History stays honest and safe for everyone.

To revert an older commit, use its ID from `git log --oneline`: `git revert 8d2e5b1`.

Reverts can conflict; resolve as in Lesson 7.3 and run `git revert --continue`.

After a revert, `git push` as usual.

Real-world analogy

> Amend is fixing a typo in a letter before posting it. Revert is sending a second letter that cancels the first.

Diagram

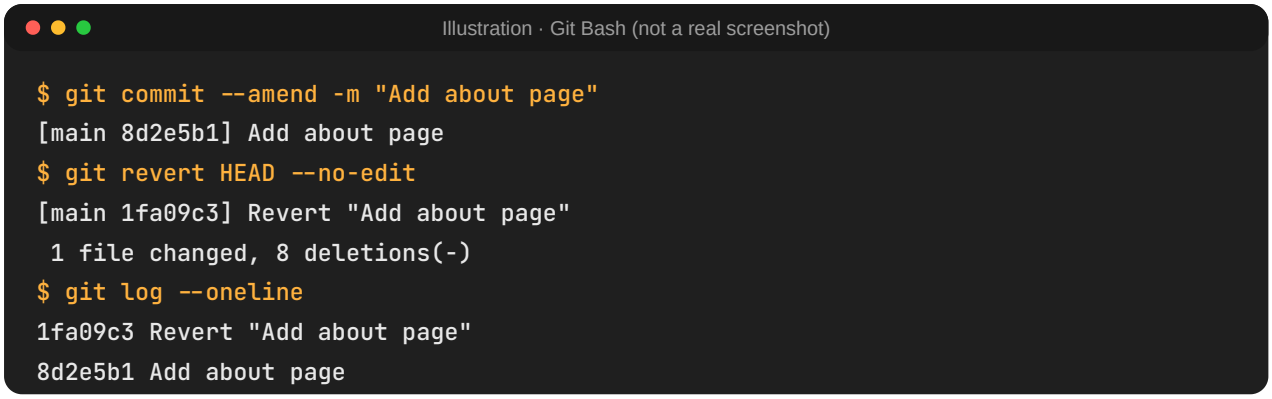


Figure 1 · Illustration: amend and revert

Commands

`git commit --amend -m "msg"`
`git revert HEAD`
`git revert COMMIT-ID`

Figure 1 shows both.

Common mistakes and solutions

Amending after push.	Use revert; amending needs a force push, which is risky.
Reverting the wrong ID.	Check `git log --oneline`.
Expecting revert to delete history.	It adds a new commit.

Practical exercise

> Make a commit with a typo in the message, amend it. Then revert your last commit.

Expected result

`git log --oneline` shows the revert commit on top.

Knowledge check

1. Which is safe for pushed commits?

- ○ git reset –hard
- ○ git revert
- ○ git commit –amend

2. What does amend change?

- ○ The last commit
- ○ All commits
- ○ The remote

3. Does revert delete history?

- ○ Only locally
- ○ No
- ○ Yes

Answers

- git revert — Revert adds a new commit.
- The last commit — It rewrites it.
- No — It adds a cancelling commit.

Summary

Amend before pushing, revert after pushing.

Key vocabulary

amend	Replace the last commit
revert	Commit that cancels another
commit ID	Short hash like 8d2e5b1

Next lesson

Lesson 10.3 — Reset, reflog and common errors.

MODULE 10 · LESSON 10.3

Reset, reflog and common errors

Reset وreflog والأخطاء الشائعة

Learning objectives

- Explain soft, mixed and hard reset.
- Recover with reflog.
- Fix five common errors.

Prerequisites

Lesson 10.2.

Introduction

Reset moves your branch backwards. It is powerful, so learn its three modes.

Explanation

`git reset --soft HEAD~1` undoes the last commit but keeps the changes **staged**.

`git reset HEAD~1` (mixed, the default) keeps the changes in files but **unstaged**.

`git reset --hard HEAD~1` undoes the commit **and discards the changes**. Use it only on local work you do not need.

Never reset commits you already pushed; use revert.

`git reflog` lists where HEAD has been, including commits that look lost. Use `git reset --hard ID` or `git switch -c rescue ID` to get back (Figure 1). Reflog is local and entries expire after some weeks.

Common errors. “Author identity unknown” → set `user.name` and `user.email`. “not a git repository” → `cd` to the project. “nothing to commit” → nothing changed or not staged. “failed to push some refs … fetch first” → `git pull` then push. “detached HEAD” → `git switch main` (or `git switch -c name` to keep the work).

Real-world analogy

> Soft, mixed and hard are three depths of rewinding a tape: keep everything ready, keep the edits, or erase.

Diagram



Figure 1 · Illustration: reset and reflog

Commands

`git reset --soft HEAD~1`
`git reset HEAD~1`
`git reflog`
`git reset --hard ID`

–hard is destructive for uncommitted work.

Common mistakes and solutions

Using –hard casually.	Prefer soft or mixed.
Resetting pushed commits.	Use revert.
Giving up on lost commits.	Look in reflog first.

Practical exercise

> Make a test commit, undo it with reset –soft, then recommit.

Expected result

The file is staged again after reset –soft.

Knowledge check

1. Which reset keeps changes staged?

- ○ –mixed
- ○ –soft
- ○ –hard

2. How do you find a “lost” commit?

- ○ git reflog
- ○ git clone
- ○ git fetch

3. “fetch first” on push means…

- ○ Reinstall
- ○ Pull before pushing
- ○ Delete repo

Answers

- –soft — Soft keeps them staged.
- git reflog — It records HEAD moves.
- Pull before pushing — GitHub has newer commits.

Summary

Reset rewinds locally, reflog rescues, and revert is for shared history.

Key vocabulary

reset	Move the branch to another commit
reflog	Log of HEAD movements
detached HEAD	HEAD on a commit, not a branch

Next lesson

> **Module 11 — Professional habits.**

MODULE 10

Practical lab 10 · Undo safely

1. Edit `index.html` and run `git restore index.html`. Check the edit is gone.
2. Edit `style.css`, run `git add style.css`, then `git restore --staged style.css`. Check with `git status`. Then restore it.
3. Create `test.txt`, commit it as `Add test file`, then run `git commit --amend -m "Add test file for practice"`.
4. Run `git revert HEAD --no-edit`.
5. Create another commit, then `git reset --soft HEAD~1` and recommit it.
6. Run `git reflog` and find your commits.

Check	Pass if…
Restore	Edits discarded as expected
Amend/revert	Log shows the revert commit
Reset/reflog	You recommitted and found it in reflog

Evaluation: 3 of 3 = pass.

Module 10 quiz (pass mark: 70%)

1. Which command discards edits in a file?

- ○ git add FILE
- ○ git push
- ○ git restore FILE

2. Which is safe on pushed commits?

- ○ git commit –amend
- ○ git revert
- ○ git reset –hard

3. reset –soft keeps changes…

- ○ Staged
- ○ Deleted
- ○ Remote

4. How do you find a lost commit?

- ○ git config
- ○ git reflog
- ○ git init

5. Push says “fetch first”. What now?

- ○ Delete the repo
- ○ Force push
- ○ git pull then push

Answers

- git restore FILE — It is irreversible.
- git revert — It adds a new commit.
- Staged — They remain staged.
- git reflog — It tracks HEAD.
- git pull then push — Get the newer commits first.

11

MODULE 11

Professional Habits

Write a README, ignore files you should not track, and keep secrets safe.

MODULE 11 · LESSON 11.1

Writing a README with Markdown

كتابة README بـ Markdown

Learning objectives

- Write Markdown basics.
- Create a useful README.md.
- Preview it in VS Code.

Prerequisites

Module 5.

Introduction

The README is the front door of your repository. GitHub shows it automatically.

Explanation

Markdown is plain text with simple symbols. `#` makes a heading, `##` a smaller one, `-` a bullet, `1.` a numbered item, `**bold**` bold text, backticks inline code, three backticks a code block, `[text](url)` a link.

A good README answers: what is this, how do I run it, and who made it. Figure 1 shows a short example.

Create `README.md` in the repository root. In VS Code press `Ctrl+Shift+V` to preview it.

Commit it: `git add README.md`, `git commit -m "Improve README"`, `git push`.

On GitHub the README appears under the file list on the repository page.

Keep it short, accurate and updated.

Real-world analogy

> The README is the sign above a shop door: it tells visitors what is inside before they walk in.

Diagram

```
# My First Project

A small website I use to learn Git and GitHub.

## Features
- Home page
- About page

## How to run
1. Clone the repository
2. Open `index.html` in your browser

## Author
[Your name](https://github.com/YOUR-USERNAME)
```

Figure 1 · Example README.md (plain text)

Commands

```
git add README.md
git commit -m "Improve README"
git push
```

Figure 1 shows a small README.

Common mistakes and solutions

A blank README.	Add at least a title and a description.
Broken formatting from missing blank lines.	Leave a blank line between blocks.
Outdated instructions.	Update it when the project changes.

Practical exercise

> Write your own README with a title, features, run instructions and author.

Expected result

GitHub shows formatted headings and a link.

Knowledge check

1. What does # make in Markdown?

- ○ A link
- ○ A heading
- ○ A comment

2. What shortcut previews Markdown in VS Code?

- ○ Ctrl+Shift+V
- ○ Ctrl+Z
- ○ Alt+F4

3. Where is README.md placed?

- ○ Only on GitHub
- ○ Repository root
- ○ Anywhere in Windows

Answers

- A heading — More # means smaller.
- Ctrl+Shift+V — It opens the preview.
- Repository root — GitHub shows the root README.

Summary

A README explains what, how and who, written in Markdown.

Key vocabulary

Markdown	Text formatting language
README	Project introduction file
Preview	Rendered view of Markdown

Next lesson

Lesson 11.2 — .gitignore.

MODULE 11 · LESSON 11.2

Ignoring files with .gitignore

تجاهل الملفات بـ .gitignore

Learning objectives

- Explain .gitignore.
- Write patterns.
- Stop tracking a file already committed.

Prerequisites

Lesson 11.1.

Introduction

Some files should never be in Git: secrets, logs, system files and downloaded dependencies.

Explanation

`.gitignore` is a text file in the repository root. Each line is a pattern for files Git should ignore (Figure 1).

Common patterns: `.env`, `*.log` (any log file), `node_modules/` (a folder), `Thumbs.db`, `.DS_Store`.

Lines starting with `#` are comments. A leading `!` re-includes a file.

Create the file, commit it, and share it with the team so everyone ignores the same things.

Important: .gitignore only affects files Git is not tracking yet. If you already committed a file, run `git rm --cached FILE`, then commit. The file stays on your disk but leaves future commits.

If it was a secret, ignoring it is not enough; it is still in history (Lesson 11.3).

Check with `git status` that ignored files no longer appear.

Real-world analogy

> .gitignore is a “do not photograph” list for a camera that otherwise photographs everything.

Diagram

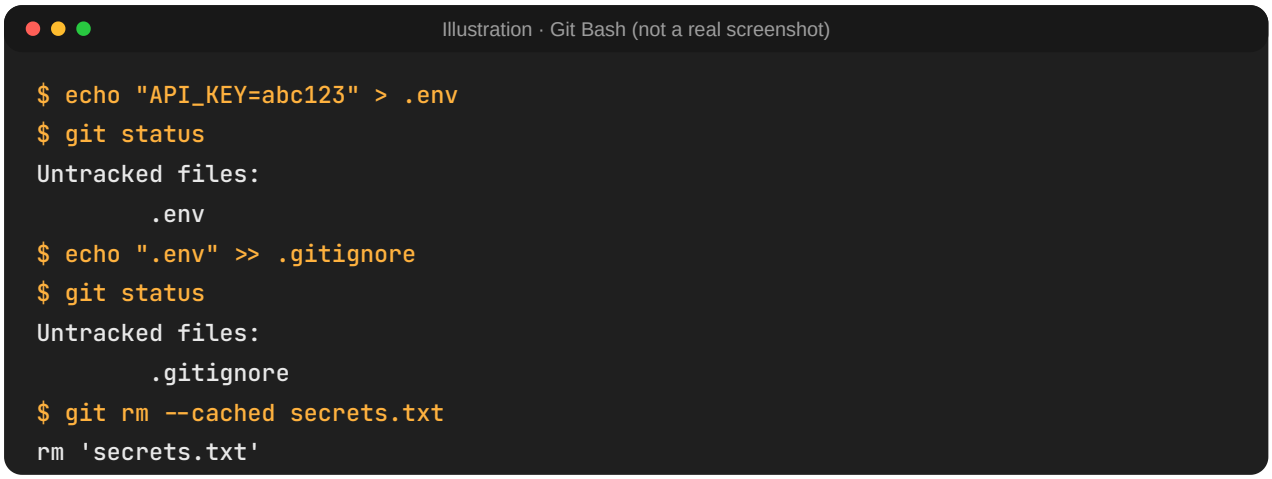


Figure 1 · Illustration: ignoring .env

Commands

`git rm --cached FILE`
`git status`

Figure 1 shows a secret file being ignored.

Common mistakes and solutions

Expecting .gitignore to untrack committed files.	Use git rm –cached.
Wrong file name such as gitignore.txt.	It must be `.gitignore` with no extension.
Ignoring too much.	Keep patterns specific.

Practical exercise

> Create .env with a fake value, ignore it, and confirm status does not list it.

Expected result

`git status` shows .gitignore but not .env.

Knowledge check

1. Where does .gitignore live?

- ○ GitHub only
- ○ Repository root
- ○ Desktop

2. Does .gitignore untrack committed files?

- ○ No
- ○ Yes
- ○ Only .env

3. What does *.log match?

- ○ Folders
- ○ All .log files
- ○ Only one file

Answers

- Repository root — Usually the root.
- No — Use git rm –cached.
- All .log files — * is a wildcard.

Summary

List patterns in .gitignore; use git rm –cached for files already tracked.

Key vocabulary

.gitignore	List of ignored patterns
pattern	Rule matching file names
git rm –cached	Untrack but keep on disk

Next lesson

Lesson 11.3 — Security and good habits.

MODULE 11 · LESSON 11.3

Security and good habits

الأمان والعادات الجيدة

Learning objectives

- Keep secrets out of Git.
- React to a leaked token.
- Adopt commit and branch habits.

Prerequisites

Lesson 11.2.

Introduction

Git remembers everything, so mistakes with secrets live forever unless you act.

Explanation

Never commit passwords, API keys, access tokens, private keys or `.env` files.

If you leaked one: first **revoke or rotate** it at the provider (for a GitHub token: Settings → Developer settings → Personal access tokens). Removing it from the latest commit is not enough, because history still has it. Treat it as public.

Turn on **two-factor authentication** on GitHub (Settings → Password and authentication).

This course uses HTTPS with Git Credential Manager, so you never store a password in a file.

Commit messages: short, imperative, specific — "Fix navigation spacing", not "changes".

Habits: pull before you start, commit small, one purpose per branch, review the diff before committing, never force-push shared branches, and keep main working.

Private repositories hide code from the public, but do not rely on them to protect secrets.

Real-world analogy

> A leaked key is a house key copied: changing the lock (rotating) is the real fix, not hiding the copy.

Diagram

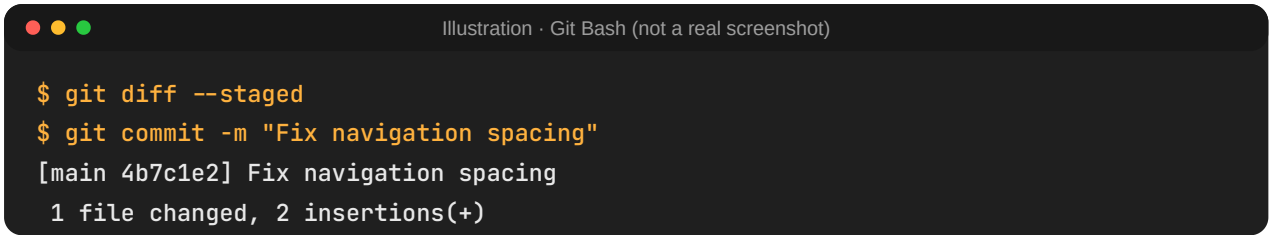


Figure 1 · Illustration: review before committing

Commands

No new commands. Revisit `git diff` and `git status` before each commit.

Common mistakes and solutions

Deleting the secret and calling it done.	Revoke it.
Using “update” as every message.	Say what changed.
Force-pushing main.	Avoid it; use revert.

Practical exercise

> Write five good commit messages for changes you have made in this course.

Expected result

Each message is under 72 characters and describes one change.

Knowledge check

1. First action when a token leaks?

- ○ Ignore it
- ○ Revoke or rotate it
- ○ Delete the repo only

2. Which is a good commit message?

- ○ Fix navigation spacing
- ○ stuff
- ○ asdf

3. What should 2FA protect?

- ○ Windows wallpaper
- ○ Your GitHub account
- ○ Your monitor

Answers

- Revoke or rotate it — History keeps it.
- Fix navigation spacing — Be specific.
- Your GitHub account — It blocks stolen passwords.

Summary

Keep secrets out, rotate leaks, enable 2FA, and keep commits clear and small.

Key vocabulary

Token	Secret used to authenticate
2FA	Second proof of identity
Rotate	Replace a secret with a new one

Next lesson

Module 12 — Final project.

MODULE 11

Practical lab 11 · README, .gitignore and safety

1. Create a branch: `git switch -c docs/readme`.
2. Rewrite README.md with a title, description, features, how to run and author link.
3. Create `.gitignore` with `.env`, `*.log`, `Thumbs.db`.
4. Create `.env` with `TEST=1` and check that `git status` does not list it.
5. Commit with clear messages, push, and open a pull request. Merge it.

Check	Pass if…
README	Renders with headings and a link on GitHub
.gitignore	.env is not in the repository
Messages	Specific and short

Evaluation: 3 of 3 = pass.

Module 11 quiz (pass mark: 70%)

1. What is README.md for?

- ○ Storing passwords
- ○ Running Git
- ○ Introducing the project

2. Which file lists ignored patterns?

- ○ index.html
- ○ .gitignore
- ○ .env

3. You committed .env earlier. What helps?

- ○ git rm –cached .env then rotate secrets
- ○ Nothing
- ○ Rename it

4. A good commit message is…

- ○ Very long
- ○ Specific and short
- ○ Empty

5. What does 2FA add?

- ○ Faster push
- ○ More storage
- ○ A second proof of identity

Answers

- Introducing the project — It is the front door.
- .gitignore — It lives in the root.
- git rm –cached .env then rotate secrets — And rotate the secret.
- Specific and short — Say what changed.
- A second proof of identity — It protects your account.

12

MODULE 12

Final Project: Portfolio Website

Use everything you learned to build, review and publish a portfolio website with Git and GitHub.

MODULE 12 · LESSON 12.1

Plan and start the project

التخطيط وبدء المشروع

Learning objectives

- Read the project brief.
- Create the repository.
- Plan branches and tasks.

Prerequisites

Modules 1–11.

Introduction

You will build a small portfolio website and manage it like a real team project.

Explanation

Brief. A one-page site with four sections: Home, About, Projects, Contact. Plain HTML and CSS; no frameworks.

Repository. Create `portfolio-website` on GitHub (public, with no README), then run the commands in Figure 1. Use `git init -b main` so the first branch is called main.

Add a starter structure: `index.html`, `style.css`, `README.md`, `.gitignore`.

Plan. Write tasks as branches: `feature/home`, `feature/about`, `feature/projects-section`, `feature/contact`, `docs/readme`.

Rules. Never commit to main after the first push; every change goes through a pull request; commit messages are specific.

You can use GitHub **Issues** to list tasks and write `Closes #1` in a PR to close one.

What the website needs. A navigation bar, a hero section with a heading and a button, an About section, a Projects section with cards, a Contact section, and a footer. Make it responsive with a viewport meta tag and a flexible grid. The folder `final-project/starter` has the empty skeleton and `final-project/solution` has one complete example to compare with afterwards.

Real-world analogy

> You are the architect and the site manager at once: first the plan, then the building.

Diagram



Figure 1 · Illustration: starting the repository

Commands

`git init -b main`
`git remote add origin URL`
`git push -u origin main`

Figure 1 shows the setup.

Common mistakes and solutions

Creating the repo with a README and then pushing a different history.	Create it empty, or clone it instead.
Starting without a plan.	List branches first.
Working on main.	Branch for every task.

Practical exercise

> Create the repository and push the skeleton.

Expected result

GitHub shows your files and a single commit on main.

Knowledge check

1. How many sections does the brief require?

- ○ Ten
- ○ Four
- ○ One

2. Where do changes go after the first push?

- ○ Through branches and PRs
- ○ Directly to main
- ○ Nowhere

3. Which command names the first branch main?

- ○ git add main
- ○ git init -b main
- ○ git clone main

Answers

- Four — Home, About, Projects, Contact.
- Through branches and PRs — Team workflow.
- git init -b main — It avoids later renaming.

Summary

Create an empty repo, push a skeleton, and plan tasks as branches.

Key vocabulary

Skeleton	Minimal starting structure
Issue	Task or bug tracker item
Brief	Project requirements

Next lesson

Lesson 12.2 — Build with branches and pull requests.

MODULE 12 · LESSON 12.2

Build with branches and pull requests

البناء بالفروع وطلبات الدمج

Learning objectives

- Build each section on its own branch.
- Open and merge PRs.
- Keep history clean.

Prerequisites

Lesson 12.1.

Introduction

Repeat one loop for each task: branch, build, commit, push, pull request, merge, pull.

Explanation

The loop (Figure 1): `git switch main`, `git pull`, `git switch -c feature/projects-section`.

Edit the files in VS Code and look at the page in your browser.

Commit in small steps, such as “Add projects heading” and “Style project cards”.

Push, open a pull request with a clear description, review **Files changed**, and merge.

Pull main and delete the branch locally.

Do this for at least **three** branches and **two merged pull requests**. Deliberately create one conflict in a safe branch and resolve it if you want extra practice.

Add a `.gitignore` and a useful `README.md` (Module 11).

Real-world analogy

> Each branch is a small renovation: you finish one room before starting the next.

Diagram



Figure 1 · Illustration: one task loop

Commands

`git switch -c feature/NAME`

`git add .` or specific files

`git commit -m "message"`

`git push -u origin feature/NAME`

Common mistakes and solutions

One giant branch for the whole site.	Split by section.
Merging with failing page.	Open the page and test first.
Forgetting to pull before the next branch.	Always pull main.

Practical exercise

> Complete the loop for Home, Projects and Contact sections.

Expected result

GitHub shows at least two merged pull requests and 8 or more commits.

Knowledge check

1. What starts each task?

- ○ git clone
- ○ git switch main and git pull
- ○ git reset –hard

2. How many merged PRs are required?

- ○ At least two
- ○ None
- ○ Fifty

3. What do you test before merge?

- ○ Only the README
- ○ The page in the browser
- ○ Nothing

Answers

- git switch main and git pull — Branch from fresh main.
- At least two — See rubric.
- The page in the browser — Make sure it works.

Summary

Branch per task, small commits, PRs, merge, then pull main.

Key vocabulary

Loop	Repeated workflow
Merged PR	Pull request combined into main
Clean history	Readable commits

Next lesson

Lesson 12.3 — Publish and submit.

MODULE 12 · LESSON 12.3

Publish with GitHub Pages and submit

النشر والتسليم

Learning objectives

- Publish a static site.
- Verify the live URL.
- Submit the project.

Prerequisites

Lesson 12.2.

Introduction

GitHub Pages hosts your static site for free from a repository.

Explanation

On GitHub open the repository, then **Settings → Pages**.

Under **Build and deployment**, set **Source** to **Deploy from a branch**, choose **main** and **/ (root)**, and click **Save**.

Wait one or two minutes and refresh. GitHub shows the address: `https://YOUR-USERNAME.github.io/portfolio-website/` .

Open the address and check all links and the CSS. Paths must be relative, such as `style.css` .

Make a fix by branch and PR; after you merge, the site updates automatically within minutes.

Submit: repository URL, live URL, and a short note listing your branches and PRs. Check the rubric before you submit.

GitHub Pages on a free account requires a public repository. Check current GitHub plan limits if you need a private one.

Real-world analogy

> Publishing is opening the shop doors: the work you did behind the scenes is now visible.

Diagram

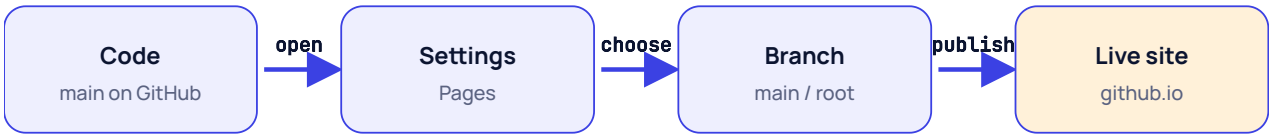


Figure 1 · Publishing path

Commands

No new Git command. Pages is configured on GitHub.

Figure 1 shows the publishing path.

Common mistakes and solutions

Page shows 404.	Wait a few minutes and check branch and folder settings, and that index.html is in the root.
Broken CSS.	Use relative paths and exact letter case.
Private repository on a free plan.	Make it public for Pages.

Practical exercise

> Publish the site and send the two URLs.

Expected result

The live URL shows your portfolio.

Knowledge check

1. Where do you enable Pages?

- ○ VS Code Extensions
- ○ Settings → Pages
- ○ Git Bash

2. Which file must be in the root?

- ○ index.html
- ○ notes.txt
- ○ .env

3. What do you submit?

- ○ Nothing
- ○ Repo URL and live URL
- ○ Only a screenshot

Answers

- Settings → Pages — It is a repository setting.
- index.html — It is the home page.
- Repo URL and live URL — Plus a short note.

Summary

Enable Pages from main/root, verify the live URL, and submit both URLs.

Key vocabulary

GitHub Pages	Free static hosting from a repository
Live URL	Public address of the site
Static site	HTML/CSS/JS files without a server

Next lesson

> **You finished the course.** Check the rubric and celebrate.

MODULE 12

Capstone lab · Portfolio website with Git and GitHub

Build the project from Lessons 12.1–12.3. Starter files are in `final-project/starter`; a sample solution is in `final-project/solution`.

1. Create the repository and push the skeleton.
2. Build Home, About, Projects and Contact on separate branches.
3. Open and merge at least two pull requests.
4. Add README.md and .gitignore.
5. Publish with GitHub Pages.
6. Submit both URLs.

Grading rubric

Criterion	Points	Full marks when…
Repository and structure	15	Clear names, index.html, style.css, README, .gitignore
Commit history	20	8+ commits with specific messages
Branches and pull requests	25	3+ branches, 2+ merged PRs with descriptions
Website content	20	Home, About, Projects, Contact work and look tidy
GitHub Pages	10	Live URL opens with working CSS
README and safety	10	README explains the project; no secrets committed

Total 100 · pass mark 70.

Module 12 quiz (pass mark: 70%)

1. What do you submit?

- ○ Only your password
- ○ Nothing
- ○ Repository URL and live URL

2. Which GitHub setting publishes the site?

- ○ Profile → Photo
- ○ Settings → Pages
- ○ Settings → Billing

3. Where should work be done?

- ○ On branches, merged by PR
- ○ Only on main
- ○ Outside Git

4. A 404 on the live URL — first check?

- ○ Delete the repo
- ○ Branch, folder and index.html in root
- ○ Reinstall Windows

5. What must never be committed?

- ○ index.html
- ○ README
- ○ Secrets and tokens

Answers

- Repository URL and live URL — Plus a short note.
- Settings → Pages — Choose main / root.
- On branches, merged by PR — Team workflow.
- Branch, folder and index.html in root — Also wait a few minutes.
- Secrets and tokens — Keep them out.

APPENDICES

Resources

Cheat sheet, glossary, troubleshooting, final project and screenshot checklist.

RESOURCE

Git cheat sheet

Windows · Git Bash · VS Code

Setup

Command	What it does
`git --version`	Check that Git is installed
`git config --global user.name "Your Name"`	Set your name
`git config --global user.email "you@example.com"`	Set your email (use your GitHub email)
`git config --global --list`	Show your settings

Start

Command	What it does
`git init -b main`	Create a repository with branch main
`git clone URL`	Copy a remote repository
`git status`	Show changed, staged and untracked files

Snapshot

Command	What it does
`git add FILE`	Stage one file
`git add .`	Stage everything in this folder and below
`git add -A`	Stage all changes in the whole repository
`git commit -m "message"`	Save a snapshot with a message
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes

History

Command	What it does
`git log --oneline`	Short history, one line per commit
`git show COMMIT`	Show one commit

Branches

Command	What it does
`git branch`	List local branches
`git switch -c NAME`	Create and switch to a branch
`git switch NAME`	Switch branch
`git merge NAME`	Merge NAME into the current branch
`git merge --abort`	Cancel a conflicted merge
`git branch -d NAME`	Delete a merged branch

Remote

Command	What it does
`git remote add origin URL`	Connect a remote named origin
`git remote -v`	Show remotes
`git push -u origin main`	First push, set upstream
`git push`	Push to the upstream
`git fetch`	Download remote changes without merging
`git pull`	Fetch and merge

Undo

Command	What it does
`git restore FILE`	Discard edits in a file (irreversible)
`git restore --staged FILE`	Unstage a file
`git stash push -m "msg"`	Set edits aside
`git stash pop`	Bring stashed edits back
`git commit --amend -m "msg"`	Replace the last commit (not pushed)
`git revert COMMIT`	Make a commit that cancels another
`git reset --soft HEAD~1`	Undo last commit, keep staged
`git reflog`	Show where HEAD has been
`git rm --cached FILE`	Stop tracking a file, keep it on disk

RESOURCE

Glossary

Term	Meaning
Repository	A project folder tracked by Git, with its history.
Commit	A saved snapshot of staged changes with a message.
Staging area	Where you choose changes for the next commit.
Working directory	The files you see and edit.
Branch	A movable pointer to a commit.
main	The default branch name in this course.
HEAD	Pointer to the branch or commit you are on.
Merge	Combine one branch into another.
Fast-forward	A merge done by moving a branch label ahead.
Merge conflict	Same lines changed differently; you must choose.
Remote	A copy of the repository hosted elsewhere, such as GitHub.
origin	The default name of the main remote.
Upstream	The remote branch your local branch tracks.
Push	Upload local commits to a remote.
Pull	Fetch and merge remote commits.
Fetch	Download remote commits without merging.
Clone	Copy a remote repository to your computer.
Fork	Your own GitHub copy of someone else's repository.
Pull request (PR)	A proposal to merge a branch, with review.
Issue	A task, bug or idea tracked on GitHub.
Tracked file	A file Git already knows.
Untracked file	A new file Git is not tracking yet.
.gitignore	File listing patterns Git should ignore.
README	File that introduces the project.

Markdown	Simple text formatting language.
Stash	Temporary storage for uncommitted edits.
Revert	Commit that cancels an earlier commit.
Reset	Move a branch to another commit.
Reflog	Local log of where HEAD has been.
Detached HEAD	HEAD on a commit rather than a branch.
HTTPS	Web protocol used by this course to talk to GitHub.
Git Credential Manager	Windows helper that signs you in to GitHub.
Token	Secret used to authenticate.
2FA	Second proof of identity at sign-in.
GitHub Pages	Free static hosting from a repository.
Source Control	VS Code panel for Git.
Sync Changes	VS Code button: pull then push.
Diff	Difference between two versions.
Version control	Recording changes so you can review and restore them.

RESOURCE

Troubleshooting guide

“git” is not recognized

Cause: Git is not installed or the terminal was opened before installing.

> **Fix:** Install Git for Windows, close and reopen Git Bash or VS Code, then run git –version.

Author identity unknown

Cause: user.name and user.email are not set.

> **Fix:** Run git config –global user.name "Your Name" and user.email "you@example.com".

fatal: not a git repository

Cause: You are outside the project folder or never ran git init.

> **Fix:** cd into the project folder; run git init if it is a new project.

nothing to commit

Cause: No changes, or changes are not staged.

> **Fix:** Edit a file or run git add first; check git status.

remote origin already exists

Cause: You already added origin.

> **Fix:** Use git remote -v; fix with git remote set-url origin URL.

Repository not found

Cause: Wrong URL, or you are signed in to another account.

Fix: Check the URL on GitHub; sign in again through Git Credential Manager.

Authentication failed

Cause: Old or wrong saved credentials.

Fix: Open Windows Credential Manager, remove github.com entries, then push again and sign in.

failed to push some refs (fetch first)

Cause: GitHub has commits you do not have.

Fix: git pull, resolve conflicts if any, then git push.

Your local changes would be overwritten

Cause: Uncommitted edits block a switch or pull.

Fix: Commit or git stash first, then retry.

CONFLICT (content)

Cause: Two changes touched the same lines.

Fix: Edit the file, remove markers, git add, git commit. Or git merge –abort.

detached HEAD

Cause: You checked out a commit, not a branch.

Fix: git switch main, or git switch -c name to keep the work.

Push rejected on main (protected)

Cause: The branch is protected.

Fix: Push a feature branch and open a pull request.

I committed a secret

Cause: A password or token is in history.

Fix: Revoke or rotate it first; then remove it from the code and add it to .gitignore.

.gitignore does not work

Cause: The file was already tracked.

Fix: git rm –cached FILE and commit.

GitHub Pages shows 404

Cause: Pages not enabled yet, wrong folder, or no index.html in root.

Fix: Settings → Pages: main / root; wait a few minutes; check index.html.

VS Code does not show Source Control changes

Cause: Opened the wrong folder.

Fix: File → Open Folder and choose the folder that contains .git.

CAPSTONE

Final project and rubric

Brief

Build a one-page portfolio website with Home, About, Projects and Contact sections using plain HTML and CSS. Manage it with Git and GitHub like a team project and publish it with GitHub Pages.

Requirements

- Repository named `portfolio-website` with index.html, style.css, README.md and .gitignore.
- At least 8 commits with specific messages.
- At least 3 feature branches and 2 merged pull requests with descriptions.
- No secrets in the repository.
- Live GitHub Pages URL.

Starter and solution

Starter files: `final-project/starter`. A sample solution: `final-project/solution`. Use the solution only to compare after you finish.

Rubric (100 points, pass mark 70)

Criterion	Points	Full marks when…
Repository and structure	15	Clear names, index.html, style.css, README, .gitignore
Commit history	20	8+ commits with specific messages
Branches and pull requests	25	3+ branches, 2+ merged PRs with descriptions
Website content	20	Home, About, Projects, Contact work and look tidy
GitHub Pages	10	Live URL opens with working CSS
README and safety	10	README explains the project; no secrets committed

RESOURCE

Screenshot checklist

The course uses clearly labeled illustrations instead of real screenshots. To replace them, capture each item on Windows with an English interface and the dark theme, then send the files with the lesson number. Annotations (numbered callouts) are added afterwards.

#	Lesson	Capture
1	2.1	Git for Windows download page (git-scm.com)
2	2.1	Git installer: Select Components and default editor screens
3	2.1	Git Bash showing git –version
4	2.1	VS Code download page and first launch
5	2.1	VS Code integrated terminal with Git Bash profile
6	2.2	GitHub sign-up page
7	2.2	GitHub two-factor settings page
8	2.3	git config –global –list output
9	3.1	File Explorer with hidden items enabled showing .git
10	3.2	git init and git status output
11	3.3	VS Code Explorer with U-marked untracked files
12	4.1	git status with staged and unstaged files
13	4.2	git commit output
14	4.3	git log –oneline with three commits
15	5.1	GitHub New repository page
16	5.2	First git push output and sign-in window
17	5.3	Repository page after push (file list, branch selector, commits)
18	6.1	VS Code Source Control panel with changes
19	6.2	VS Code commit message box and Sync Changes button
20	6.3	VS Code branch picker (status bar)
21	7.3	VS Code merge conflict with Accept buttons and Merge Editor
22	8.1	GitHub Code button with HTTPS URL
23	9.2	GitHub Compare & pull request banner and PR form
24	9.3	Merge pull request button and Delete branch

25	11.1	README rendered on GitHub
26	12.3	GitHub Settings → Pages configuration
27	12.3	Published portfolio site (live URL)